Sakala AI POC — Complete System Handoff for the Jarvis Build

As of 14 Oct 2026 · Written for the R&D team that will build the voice-first "Jarvis" version of the POC

1. How to read this document

This is the single source of truth for the Sakala AI POC as it stands on 14 Oct 2026, written so an R&D team can rebuild it as a voice-first "Jarvis" without asking the original builders anything. Everything below was verified against the running code in `D:\Sakala POC\app`, the real data files, and the official Sakala/iPGRS sources; anything assumed rather than verified is marked **ASSUMED**.

What already exists. A working FastAPI + SQLite application on port 8020 with three profiles (Citizen on WhatsApp, Officer, Chief Secretary), a real AI layer on the team's own H200 gateway, 9,999 real iPGRS grievance records, learned routing rules that match the real officer in 97.6% of cases, a two-way WhatsApp engine, deadlines and auto-escalation, a leadership dashboard with a live Karnataka map, charts, an AI question-answering mode, and 14 implemented guardrails. It was demoed to the Chief Secretary on 14 Oct 2026. Her verdict: correct and useful, but "no wow factor".

What is to be built. The same system driven by a Jarvis-style voice agent across all three profiles: no clicking, no typing. The citizen speaks, the officer speaks, the Chief Secretary speaks; every screen updates live; the visuals are futuristic. Decided constraints: **English only (no Kannada voice)** and **every model runs on the team's existing H200 gateway** using the existing `.env` file (Qwen models). Section 17 is the Jarvis brief; sections 2 to 16 are the system Jarvis must drive.

Reading order by role

If you are	Read first	Then
Building the voice agent	17 (Jarvis brief), 15 (API reference), 9 to 11 (what each profile can do)	6 to 8 (how the AI decides), 13 (guardrails)
Building the visuals	11 (CS screens), 16 (UI build system, design tokens), 9 (WhatsApp stage)	17
Understanding the domain	2 (government), 3 (claim), 4 (data), 5 (hierarchies)	12 (deadlines and law)
Operating or testing it	14 (code architecture), 16 (run, reset, tests)	15

Conventions. Code names are in `monospace` and are exact. Money is in rupees. Dates are IST. "Case" means either a grievance (iPGRS style, escalation ladder L1 to L3) or a Sakala application (legal clock, compensation); the two never mix and section 5 explains why. "Gateway" means the team's internal OpenAI-compatible vLLM server at `https://10.48.242.4/...`, reachable only on the office LAN or VPN.

Companion material the team lead will hand over separately: `Sample grievance cases.xlsx` (the real data), the three Sakala 2025 performance workbooks, `Sakala_POC_Scope.pdf`, `Sakala Use cases and Important Links.pdf`, `Sakala_AI_POC_Final_Use_Cases.pdf`, `Sakala_AI_POC_Architecture.png/.pdf`, and the code folder itself with its `CLAUDE.md` handoff file.

2. The government context

Karnataka runs two parallel citizen-facing accountability systems, and the POC sits exactly where they meet: **Sakala** (a legal right to receive a government *service* on time) and **iPGRS / Janaspandana** (the state grievance system for *complaints*). In June 2026 both were put under one new **Praja Seva Department**, and on 11 Sep 2026 the government announced it would use AI to route grievances because citizens and counter operators keep choosing the wrong category. That announcement is the POC's reason to exist.

2.1 Sakala: the right to a service on time

The **Karnataka Sakala Services Act, 2011** (Act 1 of 2012, in force 2 Apr 2012; originally titled "Guarantee of Services to Citizens", which is why every application number is still called a **GSC number**) gives every citizen a legally guaranteed delivery time for 1,446 services across 114 departments and organisations, with about 43 crore (430 million) receipts to date. Citizens apply through Seva Sindhu or department portals; Sakala tracks the application and handles appeals. Call centre 080-4455 4455. Ticker tagline: "ಇಂದು… ನಾಳೆ… ಇನ್ನಿಲ್ಲ — ಹೇಳಿದ ದಿನ ತಪ್ಪೊಲ್ಲ" ("Today… tomorrow… no more; the promised day will not be missed").

Provision	What it says	How the POC uses it
Section 9	Compensatory cost of ₹20 per day of delay, capped at ₹500 per application	`sla.compensation()` computes exactly this on every late Sakala application
Section 11	The Competent Officer recovers the cost from the officer at fault by notice (Form C), 7 days to reply; under s.11(4) (2014 amendment) the HRMS debit note is posted within 30 days	Officer drawer: "CO compensation" → Form C → mock HRMS debit
Section 12 and 13	1st appeal to the Competent Officer within 30 days; 2nd appeal to the Appellate Authority within 60 days	Each Sakala application carries a Designated Officer (DO), Competent Officer (CO) and Appellate Authority (AA)
Section 14(4)	More than 7 defaults by one officer → departmental enquiry	CS screen "E-4 defaulters": officers with more than 7 overdue cases get a Form E-4 show-cause draft
Rules 2012	An incomplete application gets no delivery date (the clock starts only when complete); public holidays excluded; compensation is paid from imprest and recovered later; Form E-4 is the show-cause notice	Stage 4 (completion agent) exists so the clock only starts on a complete request
2026 Amendment Bill (LA Bill 28/2026)	Passed the Assembly 27 Aug 2026; Council passage and assent unconfirmed. Adds s.13-A (government revision of appeal orders) and s.13-B (cost on vexatious complainants)	Mentioned in the roadmap only

2.2 iPGRS / Janaspandana: the grievance system

iPGRS (Integrated Public Grievance Redressal System, branded Janaspandana) went live on 1 Nov 2021 under High Court order 41189/2017 and a DPAR government order. Content is owned by the Centre for e-Governance; the software (.NET / MSSQL, hosted at the Karnataka State Data Centre) was built by the Centre for Smart Governance. The State Level Review Committee is chaired by the Chief Secretary. Helpline **1902**.

- **Routing today.** A "Grievance" goes to the **Last Mile Functionary (LMF, also called L1)** by location, then escalates to L2 and L3. About 17,000 LMFs are mapped down to Gram Panchayat level. A "General / PrajaSeva Request" goes to a Secretary or Head of Department instead.

- **Process rules.** An Action Taken Report (ATR) is mandatory before closing. The citizen can escalate twice. Per-level day limits are not public; a 2023 CM directive said 15 days. The POC's `SLA_DAYS` of 7 / 5 / 3 are therefore **ASSUMED**.
- **The form.** Six mandatory dropdowns (department, line department, service, grievance name, two sub-categories), a 750-character description, PIN code captured. These dropdowns are exactly what citizens and operators get wrong, and exactly what the AI now fills.
- **Excluded topics.** RTI, sub-judice matters, religious matters, suggestions, employee service matters. The POC's "wrong-door detector" sends these elsewhere.

2.3 Praja Seva Department and the Wednesday review

- Cabinet approval 20 Jun 2026; it merges the Sakala Mission, Public Grievances and Administrative Reforms. Principal Secretary Harsha Gupta posted 3 Jul 2026.
- **Praja Seva Abhiyana:** Saturday grievance camps, launched 5 Sep 2026 at Kudligi; Grama One / Karnataka One counters digitise the paper petitions. In the real data 41% of all grievances come from these camps.
- **Wednesday review:** every Wednesday the Tahsildar and Executive Officer review 9 to 11, the Assistant Commissioner 11 to 1, the Deputy Commissioner 3 to 5. The POC's "Wednesday Review Pack" is generated for exactly these four levels.
- CM D.K. Shivakumar (since 3 Jun 2026); e-Governance minister Priyank Kharge. iPGRS 2.0 was announced in June 2026.

2.4 Why the state wants AI routing

The real data shows the problem the government named: 39% of complaints filed through the mobile app are stuck in "Registered & Sent for Scrutiny" because the category was wrong or unclear, and about 349 RDPR "General" complaints sit on one state-level KRIDL desk instead of with a field officer. The Sakala 2025 reports show 4.45% of 35.2 million services delivered late and 7.42% rejected, with the rejection rate rising from about 6% to 8.5% through the year. Both systems are strong on tracking and weak at the first step: getting the complaint to the right person. That first step is what the POC automates.

Decision-maker. The demo audience is the **Chief Secretary, Shalini Rajneesh** (addressed as "Madam"). She chairs the iPGRS review committee, so the leadership profile is built around her Wednesday review.

3. The problem and the core claim

The claim in one paragraph. A citizen files one complaint or service request, from any channel, in Kannada or English. The system completes it, validates it, and routes it to the right department **and the exact field officer for that village or Gram Panchayat**, choosing only from the government's official masters and showing its reason. It then makes sure that officer acts: reminders, automatic escalation up the chain, Sakala legal deadlines and compensation where the law applies, and a live leadership view for the Chief Secretary.

Why "exact officer" and not "right department". The build lead (Pranay) explicitly rejected routing that stops at district level. In the real data the officer is named per Gram Panchayat, for example `Panchayat Development Officer(PDO) – PIRIYAPATNA – CHAPPARADAHALLI`. Routing that stops higher just moves the sorting problem to a clerk. The POC proves it can reach the named person: given the real service and location, it picks the same officer iPGRS used in **97.6% of 673 real Mysuru cases**; from raw complaint text alone it reaches the exact officer in **92.5%** of a 40-case live test.

Scope deliberately kept small so it can be real.

Dimension	Pilot scope	Why
Departments	Revenue, Rural Development & Panchayat Raj (RDPR), Food & Civil Supplies	Together 5,204 of the 9,999 real rows; they cover land records, water, roads, housing, pensions, ration cards
District	Mysuru	1,734 real rows, 894 in the pilot departments
Taluks	Piriyapatna, Hunsur, Mysuru	Piriyapatna alone has 721 pilot-department rows, so the demo has depth
Channels	WhatsApp (built), and the same engine for the 1902 call centre, Saturday camps, CSC / Grama One counters and the app	One pipeline, many doors
Profiles	Citizen, Officer, Chief Secretary	Mirrors who touches a grievance

What "done" looked like at the demo. Three use-case families were shown end to end: intake (completion, photo, voice, privacy, wrong-door), routing (understanding, exact officer, scrutiny desk, vulnerability fast-lane, accuracy proof), accountability (three-level escalation, Sakala clock, reminders, HRMS mock, ATR auditor, GSC bridge) and leadership (dashboard, map, historical intelligence, Wednesday pack, hotspots, AI mode, Sakala tracker, early warning). Six production-phase screens (AI callback, Namma Seva rating, live integrations, SMS/WhatsApp sending, learning loop, citizens-as-sensors) are front-end only and badged **PRODUCTION PHASE**.

The team's framing is "Neural AI Governance": connect the existing government systems (iPGRS, Sakala, Seva Sindhu, HRMS) rather than replace them. `chat.handle()` is written so a real WhatsApp Business webhook can call it directly, and the API is the integration surface a real iPGRS 2.0 could consume.

4. The real data

Everything the POC knows about categories, officers, villages and workload comes from one spreadsheet of 9,999 real grievances plus three Sakala performance workbooks. No master list was typed by hand; the ingest script (`server/ingest.py`) derives all masters from the data, which is why the demo can say "this is what iPGRS actually does".

4.1 `Sample grievance cases.xlsx`: 9,999 iPGRS grievances, 3 to 9 Oct 2026

Columns (17): G.Id, Grievance Description, Department, Line Department, Service Name, Grievance Name, Grievance SubCategory1, Grievance SubCategory2, Service Type (Sakala / Non Sakala), District (upper case, e.g. `MYSURU`), Taluk, GP/Village/Hobli, Created Date, Status, Officer Role (LMF / L2 / L3), Pending with / Closed By, Grievance Source. Category columns hold "English/ಕನ್ನಡ" pairs. Descriptions are about 77% Kannada, 17% mixed, median 258 characters; 788 rows are repeated camp templates.

Dimension	Distinct values	Notes
Departments	38	Revenue 2,543 rows, RDPR 2,384, Urban Development 1,147, Housing 882, Women & Child 418, Energy 401, Food & Civil Supplies 277
Line departments	195	e.g. Revenue → Land Records, RDPR → Panchayat Raj Engineering, RDWS
Services	703	top: RTC/Pahani 509, Housing 487, Dispute Cases 361, RDPR "General" 349, Drinking Water 235
Grievance names	838	the finest official category
Districts / taluks	31 / 235	Mysuru 1,734 rows; Piriyapatna taluk alone 1,358
Officers (Pending with / Closed by)	2,956	named "Designation-Place", e.g. `Tahsildar-Grade 1-Piriyapatna`

Status: InProcess 6,166 · Closed by Officer 3,427 · Registered & Sent for Scrutiny 342 · Resolved (Feedback Taken) 64. **Officer role:** LMF 9,224 (92%) · L2 653 · L3 122. L2 and L3 appear even on same-day rows, so the role partly reflects direct assignment, not only escalation. **Service type:** Non Sakala 8,611 · Sakala 1,388 (14%).

Source channel: Praja Seva Abhiyana camps 4,120 (41%) · Citizen Service Centre 2,402 (24%) · iPGRS website 1,904 (19%) · Mobile App 877 (9%) · iPGRS call centre 507 · RDPR call centre 134 · Department officers 30 · ZP CEO 14 · HF-QR code 11. The Saturday 3 Oct camp alone produced 5,005 rows, so this is one camp week, not a statewide average.

Findings the demo leans on

- **39% of Mobile App complaints** sit in "Registered & Sent for Scrutiny" against nearly 0% for other channels: the manual triage bottleneck that AI routing removes.
- About **349 RDPR "General"** complaints sit with one state KRIDL "Superintendent" desk: a category chosen because the right one was unclear.
- **Routing is deterministic in the real data:** line department + taluk + GP maps to exactly one officer in 98.4% of rows. This is what makes exact-officer routing learnable.
- **Pilot departments** (Revenue, RDPR, Food) hold 5,204 rows. RDPR has 0 Sakala-tagged rows; 258 of the 385 Sakala-tagged pension rows are payment problems, not Sakala delays, which is why the GSC bridge (section 11) tells the two apart.

- **Mysuru district:** 1,734 rows; 894 in pilot departments; Piriyapatna 721 of those. PDOs are named per Gram Panchayat. Spellings differ between the GP column and officer names (ATTIGODU vs ATTHIGUDU, BYLAKUPPE vs BYLUKUPPE), handled by fuzzy matching.
- **Privacy:** 434 descriptions hold phone numbers and 247 hold Aadhaar-like numbers. All are masked at ingest; personal names inside Kannada text are not masked (known gap).
- **No closed date, due date or SLA days** exist in the data, so resolution time and breach figures cannot be demonstrated on real rows; the escalation demo therefore uses the demo clock and synthetic cases.

4.2 What ingest derives from it (`app/data/sakala.db`)

Derived master	Count	How
Pilot services	124	Every service seen under the 3 pilot departments; each carries a learned `role` , `role_share` , `role_support`
Grievance types	221	Grievance names under those services
Officers	88	61 PDOs (one per Gram Panchayat) + taluk officers (Tahsildar, ADLR, Sub-Registrar, AEE PRED, AE RDWS, Food Inspector) + supervisors (AC, DDLR, DD Food, EO TP, ZP CEO, DC). `seen_in_data=1` on 50; the rest are standard posts added to complete the ladders
Gazetteer	198 villages → 61 GPs → 3 taluks	Piriyapatna 160 villages / 34 GPs, Mysuru 21 / 15, Hunsur 17 / 12
Live caseload	703 real cases	Every pilot-department Mysuru grievance with a mappable officer, loaded as a real case with its real owner; plus 10 synthetic grievances and 2 synthetic Sakala applications. At go-live: 393 Pending, 318 Resolved, 3 Scrutiny; 692 at L1, 22 at L2
Masked records	9,999	`ipgrs_records.text_masked` used for nearest-case evidence and AI-mode questions

4.3 Sakala 2025 district performance reports

Three workbooks (`combined_dec 1.xlsx` , `RDPR.xlsx` , `REVENUE Performancereport 2025.xls`) each hold 12 monthly sheets, 31 districts plus a "BBMP & Other Institutions" row, with receipts, disposals, rejected, delayed, in-time, overdue, per-lakh figures, weightages (in-time 50, rejected 10, receipts 20, overdue 20), total score and rank. **All three are numerically identical in every month**; only the titles differ, so they are statewide all-department figures, not department-wise. They load into `sakala_perf` (384 rows).

- 35.2 million receipts in 2025; **4.45% delivered late**; **7.42% rejected**, rising from about 6% to 8.5% through the year (district rows only; the BBMP row is excluded because it reuses Bengaluru Urban's population).
- Overdue fell from **51,172 (Jan) to 19,129 (Dec)**, a 63% drop.
- **Bengaluru Urban is worst** (9.2% late; 6,326 overdue in December, about a third of the state). Mysuru 5.4% late, ranked 26 to 29. Best: Kalaburagi 1.9%.
- The official ranking barely separates districts (total scores 61 to 79) because it is driven almost only by in-time %. This is the basis of the "fairer scorecard" in section 11.
- Names still say "Ramanagara" (now Bengaluru South). Department-wise exports must be requested from Sakala.

5. Government hierarchies and the two ladders

A grievance climbs a three-level **escalation ladder** inside its own department (L1 → L2 → L3); a Sakala application has a legally fixed **appeal ladder** (Designated Officer → Competent Officer → Appellate Authority). The two share people but never mix: a grievance never earns Sakala compensation and an application never auto-escalates on the grievance clock. This is guardrail 14 and is enforced in `sla.insert_case`, `api.case_action` and `sla.tick`.

5.1 The grievance ladder per department (pilot, Mysuru district)

Department	L1 (Last Mile Functionary)	L2	L3
Revenue (land, certificates, pensions)	**Tahsildar Grade-1** of the taluk (Piriyapatna, Hunsur, Mysuru)	**Assistant Commissioner**: Hunsur sub-division covers Piriyapatna and Hunsur; Mysuru sub-division covers Mysuru taluk	**Deputy Commissioner, Mysuru**
Revenue: survey (podi, hadbast)	**ADLR** (Assistant Director of Land Records, taluk survey)	**DDLR Mysuru**	DC Mysuru
Revenue: registration	**Sub-Registrar** of the taluk	DC Mysuru (no intermediate post in the data)	
Food & Civil Supplies	**Food Inspector** of the taluk	**Deputy Director, Food & Civil Supplies, Mysuru**	DC Mysuru
RDPR: panchayat matters (housing, street lights, sanitation, drains)	**PDO** (Panchayat Development Officer) of the Gram Panchayat; 61 GPs	**Executive Officer, Taluk Panchayat**	**CEO, Zilla Panchayat Mysuru**
RDPR: roads, bridges, culverts	**AEE PRED** (Assistant Executive Engineer, Panchayat Raj Engineering) of the taluk	EO Taluk Panchayat	ZP CEO
RDPR: drinking water	**AE / JE RDWS** (Rural Drinking Water & Sanitation) of the taluk	EO Taluk Panchayat	ZP CEO

Officer ids follow the pattern `role_taluk[_gp]`: `tah_piriyapatna`, `ac_hunsur`, `dc_mys`, `pdo_piriyapatna_chapparadahalli`, `aee_hunsur`, `rdws_piriyapatna`, `fi_hunsur`, `dd_food`, `eo_piriyapatna`, `ceo_zp`, `adlr_mysuru`, `ddlr_mys`, `subreg_mysuru`. Each officer row has `up_id` (the next level), `data_name` (the exact iPGRS spelling, e.g. `Assitance Executive Engineer-PRED-PIRIYAPATNA`), `seen_in_data` and `n_real` (real cases in the sample). The Tahsildar of Piriyapatna alone had 262 real cases, which is why his inbox is the heaviest in the demo.

Ladder day limits (ASSUMED): `SLA_DAYS = {L1: 7, L2: 5, L3: 3}`. iPGRS publishes none; a 2023 CM directive said 15 days. When a case passes its limit at a level, `tick()` moves it to `up_id`, records "Auto-escalated" and notifies both officers and the citizen. Cases already at L3 stop escalating but stay overdue.

5.2 The Sakala appeal ladder (applications only)

Role in the Act	Who, in the pilot	Duty
Designated Officer (DO)	Tahsildar (Revenue services), Food Inspector (ration cards)	Must deliver within the notified days

Role in the Act	Who, in the pilot	Duty
Competent Officer (CO)	Assistant Commissioner (Revenue), DD Food (Food)	Hears the 1st appeal within 30 days; orders compensation and recovers it from the DO (Form C, HRMS debit)
Appellate Authority (AA)	Deputy Commissioner	Hears the 2nd appeal within 60 days

Illustrative Sakala timelines used (ASSUMED, to be confirmed against the notified list): Caste & Income Certificate 21 days, Old-age pension 45 days, New ration card 30 days. The clock starts at `complete` (when the application is complete), holidays are not yet excluded in code.

5.3 Where a case lands: the service → role rules

The officer *role* for a service is not configured; it is learned from the 9,999 rows as the role that handled that service most often (`services.role`, with `role_share` = share of real rows and `role_support` = how many rows). Examples the demo quotes:

Service	Learned role	Share	Support
RTC / Pahani	Tahsildar	99.2%	506
Khatha, Cultivation card, Old-age schemes	Tahsildar	100%	203 / 192 / 220
Dispute cases	Tahsildar	98.9%	360
Housing	PDO	98.7%	471
Maintenance of street light, sanitation	PDO	97% / 95.5%	
Provision of drinking water	AE RDWS	100%	229
Village connecting roads and bridges; village internal roads	AEE PRED	100% / 99.3%	155 / 136
Ration card	Food Inspector	100%	153
Tatkal podi, hadbast	ADLR	~98%	
RDPR "General"	EO Taluk Panchayat	fallback (real rows sat on a KRIDL state desk)	349

Where a role is taluk-level (Tahsildar, AEE, AE RDWS, Food Inspector, ADLR, Sub-Registrar) the citizen is never asked for a village; only PDO services need the Gram Panchayat. Note the client's wireframe sent drinking water to the PDO; the real data says AE RDWS in 100% of cases, so the app follows the data.

5.4 Geography

Karnataka has 31 districts (plus BBMP in Bengaluru). The pilot is **Mysuru district**, taluks **Piriyapatna, Hunsur, Mysuru**. Location resolves village → Gram Panchayat → taluk through the gazetteer (`gazetteer` table: 198 villages, 61 GPs). Kannada village names are transliterated to Latin before matching (ಹೊಸಹಳ್ಳಿ → Hosahalli). A correction worth knowing: the client's architecture image places Bylakuppe in Hunsur; it is in **Piriyapatna**. The full LGD village master is not loaded yet, so Hunsur and Mysuru taluks have few GPs; that is open item 5 in the handoff file.

6. The 8-stage pipeline

Every request, from any channel, passes the same eight stages (`pipeline.STAGES`). The citizen watches them light up in the "Behind the chat" panel; the officer and the Chief Secretary see the result. The build lead asked that this flow be kept exactly as is; Jarvis should narrate it, not change it.

#	Stage	What happens	Why it exists	Code
1	**Entry**	One WhatsApp number; the same `chat.handle()` also serves the 1902 call centre, Saturday camps, CSC / Grama One counters, the mobile app and bulk Excel upload (`src` = whatsapp, whatsapp-voice, whatsapp-photo, camp, csc, web, app, call, rdpr, upload)	Citizens already use WhatsApp; no app to install; one pipeline for every door	`chat.py` , `pipeline.process()`
2	**Privacy shield**	`privacy.mask()` replaces emails → `[EMAIL]` , 12-digit Aadhaar → `[AADHAAR]` , Indian mobiles (optional +91) → `[PHONE]` , 2–3 capitals + 8–13 digits → `[ID-MASKED]` . The AI only ever sees the masked text. GSC numbers (`RD855S260004811`) survive because a letter sits inside the digits, so the Sakala bridge can still read them	434 real texts contain phone numbers, 247 Aadhaar-like numbers	`privacy.py`
3	**AI understanding**	`pipeline.understand()` : gateway model with an enum-constrained JSON schema picks the intent and the official service id; local keyword rules override on wrong-door / out-of-scope / service-request; the offline model gives a second opinion; a tuned confidence threshold decides auto-route vs human. Output: Department → Line department → Service → Grievance name, confidence, reason, vulnerability flag, English summary, village and taluk mentioned	The government's stated problem is wrong categories; the AI must choose only from the master and explain itself	`ai.understand` , `local_ai.understand` , `pipeline.understand` (section 7)
4	**Completion agent**	Asks only for what is missing. Usually the Gram Panchayat, and only when the service is handled per GP (PDO). Taluk-level services need no village. Accepts GPS, a GP pick, a typed village (Kannada transliterated) or a village or taluk already in the text	Under the Sakala Rules an incomplete application has no delivery date; the clock must start on a complete request	`chat.grievance()` , `pipeline.locate()` , `routing.resolve_place()`

#	Stage	What happens	Why it exists	Code
5	**Validate & confirm**	Location checked against the gazetteer; `find_duplicate()` links to an open case with the same service in the same GP; the citizen confirms a summary card (Problem / Service / Location)	Nothing is filed that the citizen has not seen; duplicates are linked, not multiplied	`chat.confirm()`, `pipeline.find_duplicate()`
6	**Exact-officer routing**	`routing.route(service_id, taluk, gp)`: learned service → role, then role + taluk (+ GP) → one named officer; the escalation chain (`sla.chain`) is attached; `in_masters()` refuses anything not in the official master; the audit record is written	The named person, not the department (97.6% agreement with real iPGRS assignments)	`routing.py`, `pipeline.create_grievance()` (section 8)
7	**Deadlines & reminders**	The clock starts (`level_since = now`), the officer is notified, and the ladder is armed: 7 / 5 / 3 days per level for grievances, the legal number of days for Sakala applications. `tick()` sends due-today / due-tomorrow reminders, auto-escalates and records Sakala breaches	Accountability without anyone chasing	`sla.py` (section 12)
8	**Leadership view**	The case is live at once on the officer's inbox, the CS dashboard, the Karnataka map and in AI mode	The CS sees the system working, not a report about it	`api.py /api/state`, UI

Worked example ("road is bad in Hunsur", typed in English):

1. Entry: `src=whatsapp`.
2. Privacy: "Nothing personal to mask".
3. Understanding: qwen4-deploy returns `GRIEVANCE`, service `S17 Village Internal Roads` (RDPR › Panchayat Raj Engineering), confidence 0.9x, `taluk="Hunsur"`; local rule `road` agrees; threshold passed.
4. Completion: `taluk_in_text()` finds Hunsur; `route(17, "Hunsur", "")` says the AEE PRED is taluk-level (`need_gp=False`), so no village question.
5. Confirm: card "Village Internal Roads · RDPR · Hunsur taluk, Mysuru"; no duplicate.
6. Routing: officer `aee_hunsur` (AEE PRED Hunsur, L1); chain AEE PRED Hunsur → EO TP Hunsur → ZP CEO Mysuru; basis "AEE, PRED (roads & works) for Hunsur taluk. 99% of 'Village Internal Roads' grievances (136 cases) were handled by the AEE PRED".
7. Deadlines: 7 days, then EO TP Hunsur.
8. Leadership: case `GR-5042xx` appears in the inbox, on the map as a green dot in Hunsur, and in the CS counts.

Non-grievance outcomes at stage 3: `WRONG_DOOR` (RTI, employee matters) is answered with where to go; `OUT_OF_SCOPE` (Energy, Transport, Health, Education, Police) is named and explained as a production-phase hand-off; `SERVICE_REQUEST` offers the Sakala application flow; `UNCLEAR` or a guardrail failure creates a **Scrutiny** case for a human (always owner `eo_piriyapatna`, a known simplification).

7. The AI engine

All AI runs on the team's own H200 gateway, an OpenAI-compatible vLLM server at `https://10.48.242.4/openai/v1`, configured in `app/.env` (a copy of the KPCL `.env`). One file, `server/ai.py` (334 lines), wraps every model call; one file, `server/local_ai.py`, is the offline fallback trained on the 9,999 real rows. **For Jarvis the same gateway also serves `qwen3-asr` (speech-to-text) and `qwen3-tts` (text-to-speech), both English-capable; neither handles Kannada, which is why the voice agent is English-only.**

7.1 Models and env keys

Purpose	Env keys	Model	Used by
Language model (text + vision)	`SQL_VERIFY_BASE_URL`, `SQL_VERIFY_API_KEY`, `SQL_VERIFY_MODEL`	`qwen4-deploy`	understand, OCR, ATR check and draft, AI mode plan and answer, Kannada translation, briefs, hotspot fixes
Speech-to-text	`ASR_BASE_URL`, `ASR_API_KEY`, `ASR_MODEL`	`qwen3-asr` at `POST /audio/transcriptions` (multipart, 16 kHz WAV, `language`)	WhatsApp voice notes, AI-mode mic
Text-to-speech	`TTS_*` (`TTS_MODEL=qwen3-tts`, `TTS_API_STYLE=openai`, `TTS_VOICE=serena`, `TTS_STREAM_ENABLED=true`, a reference-voice WAV and style instructions)	`qwen3-tts`	**Not used yet**; configured and ready for Jarvis
TLS	`AI_MODEL_CA_BUNDLE_PATH` (empty → `verify=False`)		

Other keys in the file (`qwen35-9b`, `qwen3-embedding`, `qwen3-reranker`, Redis, Postgres, RAG) belong to the KPCL app and are unused here. `qwen35-9b` returns 404 on this gateway. Measured latencies: understand 0.2–0.7 s, ASR about 0.4 s. `MODEL_MAX_CONCURRENCY=24` and `GUIDED_DECODING_ENABLED=true` describe the gateway's capacity.

7.2 How a call is made (`ai.chat`)

- `POST {base}/chat/completions` with `model`, `messages`, `temperature` (0.0 by default), `max_tokens`, and `chat_template_kwargs: {"enable_thinking": false}` (Qwen's thinking mode off for speed).
- **Guided JSON:** when a schema is given, `response_format: {type: "json_schema", json_schema: {name, schema}}`. Enums inside the schema are how the model is physically prevented from inventing a category: `service_id` can only be one of the 124 catalogue ids or `NONE`.
- Post-processing strips `</think>` remnants and code fences, then `json.loads`.
- Client: one `httpx.Client` with 40 s total / 4 s connect timeout. On `ConnectError` the gateway is marked down for 60 s (`down_until`) and every caller falls back to the local model; `status()` reports `{llm, model, asr, asr_model, reason}`.
- Every call is logged to `ai_log(t, task, engine, ms, ok, detail)`; tasks: `understand`, `ocr`, `atr_check`, `atr_draft`, `ask_plan`, `ask_answer`, `translate`, `hotspots`, `pack_en`, `pack_kn`,

`historical`, `asr`, `ask_blocked`.

7.3 `ai.understand(masked_text)`: the heart of stage 3

The system prompt makes the model the intake officer of Karnataka Sakala / iPGRS and appends a **catalogue** of every pilot service with at least 3 real rows, one line each: `S109 | REV › Land Records › RTC/Pahani | e.g. RTC correction, Name mismatch…`, plus the list of non-pilot departments. Rules in the prompt: never invent a category; prefer a specific service over "General"; pensions not received → Old-Age Schemes; drinking water → Provision of Drinking Water; khata / pouthi / mutation → KHATHA or Land Mutation; transliterate the village to English without "namma" or "grama"; `vulnerable` = elderly, widow, disabled, sick, pension- or ration-dependent. Up to 5 similar real grievances (nearest neighbours from the offline model) are added as reference.

Schema (all fields required): `intent` ∈ GRIEVANCE / SERVICE_REQUEST / OUT_OF_SCOPE / WRONG_DOOR / UNCLEAR; `service_id` ∈ catalogue ids + NONE; `grievance_name`; `sakala_key` ∈ caste_income / pension / ration_new / none; `other_department`; `confidence`; `reason`; `village`; `taluk`; `vulnerable`; `vulnerable_reason`; `summary_en`; `language` ∈ kn / en / mixed.

Returned dict `u`: `engine="llm"`, `conf` (0–0.99), `why`, `summary`, `village`, `taluk_text`, `lang`, `similar`; for a grievance also `intent, dept_code, line, service, service_id, grievance, vulnerable, vuln_reason`; for a service request `sakala_key`; for out-of-scope `dept`.

7.4 `pipeline.understand(text)`: the decision logic around the model

1. Mask the text.
2. If the gateway is up: `u = ai.understand(masked)` and `lu = local_ai.understand(masked)` (the keyword-rule and offline-model opinion).
3. **Rules override.** If `lu` says WRONG_DOOR, OUT_OF_SCOPE or SERVICE_REQUEST and the model disagrees, the rule wins (`engine="llm+rules"`, shown as "Sakala AI · rule check"). Reason: the model once called a power cut "drinking water" and a certificate request a complaint; the high-precision rules are safer.
4. **Second opinion.** If the model says GRIEVANCE but the offline model says UNCLEAR and the chosen service is not among its top-3, confidence is capped at `threshold − 0.05`.
5. **Threshold.** If `conf < low_conf()` the result becomes UNCLEAR with "Confidence only NN%. Possible services: a / b / c" and goes to the scrutiny desk. `low_conf()` reads `kv.low_conf`, default 0.55; the accuracy run tunes it (section 7.6).
6. If the gateway fails, `u = local_ai.understand(masked)` (`engine="local"`, shown as "Sakala AI (offline model)").
7. `u.rules` and `u.threshold` are recorded for the audit.

7.5 The offline model (`local_ai.py`)

- **Architecture:** `TfidfVectorizer(analyzer="char_wb", ngram_range=(2,4), max_features=60000)` + `SGDClassifier(loss="modified_huber")`. Character n-grams work directly on Kannada script. Two heads: department (38 classes, non-pilot as `OTHER:<dept>`) and pilot service (124). Saved to `data/local_models.joblib`; trains in about 43 s.
- **Hold-out accuracy:** department 71.4%, exact service 39.2%, top-3 43.4% (iPGRS labels are themselves noisy). Quote the routing figures, not these.
- **Nearest real cases:** top-5 by TF-IDF cosine over the pilot rows; used as evidence in the audit and as examples in the LLM prompt.

- **Keyword rules (`RULES`)** map words to service ids, e.g. pouthi / mutation → 108, khata → 106, podi / survey → 119, rtc / pahani → 109, pension / ಸಂಧ್ಯಾ ಸುರಕ್ಷಾ → 57, ration card → 6, drinking water / borewell → 40, street light → 28, drain / culvert → 15, road / pothole → 17, house / ashraya → 25, toilet / garbage → 42, e-khata / form 9 → 24, nrega → 39, marriage registration → 53, temple → 71, crop loss → 61.
- **WRONG_DOOR:** `rti`, `information act` ("File it on RTI Online"), `transfer of my posting`. **OUT_OF_SCOPE:** power / electricity / ವಿದ್ಯುತ್ → Energy (CESC Mysuru); bus / ksrtc → Transport; hospital / doctor → Health; school / teacher → Education; police / fir → Home (call 112). **SERVICE_REQUEST:** an APPLY word (caste, income, new ration card) plus an apply verb (need, want, apply, ಬೇಕು) and no complaint word (delay, not received, ಬಂದಿಲ್ಲ). **VULNERABLE:** widow, old, elderly, disabled, pension, ration, years old, sick, pregnant and their Kannada forms.
- **Transliteration** `kn_to_latin()` turns Kannada words into Latin so village names match the gazetteer; `place_candidates()` yields English words of 4+ letters, transliterated Kannada words (with case endings `du / dalli / nalli / ge` stripped) and adjacent pairs.

7.6 Measured accuracy and the tuned threshold

`analytics.llm_eval(n)` (CS screen "Accuracy proof", `GET /api/insights/accuracy?run=1&n=40`) runs the model on n real Mysuru complaints using the text alone and compares with what iPGRS actually did: **department 97.5%, service 95%, exact officer 92.5%**, 40 cases in 8 s. It then picks the lowest threshold in 0.40–0.75 at which auto-routed cases are at least 90% right on service while at least 70% of cases still auto-route, and stores it in `kv.low_conf` (last run: 0.40, 100% auto-routed). Results live in `data/llm_eval.json`, outside the DB so a demo reset keeps them.

7.7 Other model functions

Function	Task	What it does
`read_petition(image_b64)`	`ocr`	Vision: transcribes a petition exactly (Kannada stays Kannada); for an ID or ration card describes only the type and village; returns `""` if unreadable or under 8 letters (guardrail 7)
`transcribe(wav, lang)`	`asr`	`qwen3-asr`; returns `""` under 4 letters
`check_atr(complaint, atr)`	`atr_check`	Scores an Action Taken Report 1–5 against the complaint (1 incorrect, 2 not resolved, 3 partial, 4 substantial, 5 fully resolved with evidence); `check_atr_local` is a keyword fallback
`draft_atr(case)`	`atr_draft`	3–4 factual sentences with `[DATE]` and `[DETAIL]` placeholders for the officer to complete
`ask(question, run_tool)`	`ask_plan`, `ask_answer`	AI mode: plan up to 3 data-tool calls, run them, write 2–5 sentences only from the results, else "I don't have that data."
`to_kannada(text)`	`translate`	Officer free text → simple polite Kannada, keeping numbers and names
`brief(task, instruction, facts)`	`pack_en`, `pack_kn`, `historical`	Writes HTML only from given facts
`guard_question(q)`		Refuses prompt-injection and off-topic questions (section 13)
`clean_html(html)`		Keeps only `b strong i em br p ul ol li`, strips attributes

The header never shows a model name: engine labels are "Sakala AI", "Sakala AI · rule check" and "Sakala AI (offline model)".

8. Exact-officer routing

Routing is a two-step lookup, not a model: **service → role** (learned from the data) and **role + taluk (+ GP) → one officer id** (the officer master). Because the real data is 98.4% deterministic on line department + taluk + GP, this reaches the named person reliably, and every decision carries its evidence.

8.1 `routing.route(service_id, taluk, gp="")`

1. Read `services.role`, `role_share`, `role_support` for the service.
2. `weak = role_share < 0.3` (the real rows went to a state desk, e.g. RDPR "General" → KRIDL); the pilot sends it to the department head role instead and flags `fallback`.
3. If the role is **PDO** and no GP is known: return `need_gp=True`, `officer=None`, basis "Needs the Gram Panchayat: each GP has its own PDO". The chat then asks for the GP.
4. Otherwise `officer_for_role(role, taluk, gp)`: PDO → `pdo_{taluk}_{gp}` if that officer exists, else the taluk EO `eo_{taluk}` (fallback, "No PDO for this GP in the pilot master, so the Taluk Panchayat EO receives it"); any other role → `{prefix}_{taluk}` with prefixes `tah, adlr, fi, aee, rdws, eo, subreg`.
5. Return `{officer, role, role_label, share, support, need_gp, fallback, basis}`. The basis text reads, for example: "Panchayat Development Officer for Chapparadahalli GP, Piriyapatna taluk. 99% of 'Housing' grievances (471 cases) were handled by the Panchayat Development Officer".

8.2 Location resolution (`routing.resolve_place`)

- Candidates come from `local_ai.place_candidates(text)` (English words, transliterated Kannada words, adjacent pairs).
- Each candidate (4+ letters, reduced to a–z) is compared with every gazetteer `village` and `gp` by `difflib.SequenceMatcher`; exact = 1.0; +0.03 if the taluk hint matches; +0.01 for a GP-field match. Best score **≥ 0.84** wins. Returns `{village, gp, taluk, score}`.
- `pipeline.locate()` trusts the LLM's extracted `village` only if it fuzzy-matches (≥ 0.8) a word actually present in the text. This stopped the model from inventing "Gorahalli", which had been the example in the prompt.
- Taluk-only mentions are handled by `chat.taluk_in_text()` with `TALUK_WORDS` (piriyapatna / periyapatna / ಪಿರಿಯಾಪಟ್ಟಣ; hunsur / hunasuru / ಹುಣಸೂರು; mysuru taluk / mysore taluk).
- `gp_list(taluk)` lists GPs by real volume for the WhatsApp picker (≤ 10 shown).

8.3 Guardrails at routing time (`pipeline.create_grievance`)

- `in_masters(u)`: the service id must exist and its name must match; the grievance name is snapped to one of that service's official grievance types (or "General"). A miss → scrutiny case "Guardrail: service not in the official master".
- The resolved officer must exist in `officers`; otherwise scrutiny "Guardrail: officer not in the official master".
- Case id `GR-<seq>` (sequence from 504200), status Pending, `level` = the officer's level, `level_since = created = now`, `route_basis`, `linked_to` (duplicate), `audit` JSON.
- Events: "Routed to {officer}" with note "{service} · via {src} · {engine label} · confidence N%"; "Linked to similar open request GR-…" when a duplicate exists. Notification: "New grievance GR-… ({service}) from {src}".
- Returns `{id, officer{id,name,short,level}, chain[{lv, short}], basis, fallback, days, next}`.

8.4 The audit record (`cases.audit`, guardrail 3)

Every routed case stores why it went where it went; the officer drawer shows it under "Routing evidence · how this decision was made".

Key	Content
`at`, `engine`, `model`	Demo time; `llm` / `llm+rules` / `local`; `qwen4-deploy` or `offline`
`confidence`, `threshold`, `intent`	The model's confidence, the tuned threshold it was compared with, the intent
`decision`	`{service_id, service, grievance, reason}`
`alternatives`	Other plausible services (currently always empty for auto-routed cases; filled for UNCLEAR)
`similar_real_cases`	Up to 5 `{gid, service_id, score}` nearest real grievances
`rules`	Rule overrides and confidence caps that fired, in words
`place`	`{village, gp, taluk, score, how}` with `how` = where the location came from
`route`	`{officer, role, share, support, fallback, basis}`
`human_decision`	`{by, to, note, at}` when the scrutiny desk re-routed

8.5 The proof numbers

- **Router alone:** `kv.routing_eval` = `{n: 673, agree: 657, rate: 0.976}`. For each real case with a known service and location, `route()` picked the same officer iPGRS used in 97.6%. The misses are almost all GP → EO fallbacks where the real data had a PDO the pilot master lacks (`pdo_piriyapatna_ravandoor → eo_piriyapatna` × 4) and FI → DD Food (× 4).
- **End to end from text:** department 97.5%, service 95%, exact officer 92.5% (n = 40).
- Both figures are shown on the CS "Accuracy proof" screen and the dashboard KPI "Exact-officer agreement".

9. Profile 1: the Citizen on WhatsApp

The citizen never installs anything: the whole experience is a WhatsApp chat with one government number, simulated in the browser as an iPhone. The conversation engine is `server/chat.py` (617 lines): stateless HTTP in, an ordered list of events out. A real WhatsApp Business webhook would call the same `chat.handle(session, request)`; buttons and list rows map one-to-one to WhatsApp interactive reply buttons and list messages. **For Jarvis this is the most important module: every citizen intent (raise, locate, confirm, track, reply, appeal, feedback) already exists as an action string; the voice agent only needs to pick the action and speak the result.**

9.1 What the citizen sees

- **Left column:** "Citizen · Sakala AI on WhatsApp" and a QR card (hidden below 760 px height; the column hides below 1280 px width).
- **Centre:** the phone. Height = window height minus header, clamped 420 to 820 px, so the page never scrolls (`fitWA()`).
- **Right column, "Behind the chat":** the 8 pipeline stages light up as the conversation proceeds (`stage` events), ending with a "Routed to" card that names the officer and the escalation chain.
- **Sidebar:** "Sakala on WhatsApp" (home) and "Track my requests" (the same data as `/api/state?scope=citizen`, with status lines like "📥 Officer asked for information · reply in this chat" and "❌ Rejected · you can appeal").
- The client (`tools/app_wa_client.js`) polls `POST /api/chat {action:"poll"}` every 3.5 s while the citizen screen is open; a new officer message nudges the phone, plays a soft sound and shows a toast.

9.2 Session state (`chat_sessions.state`, JSON)

Field	Meaning
`lang`	`en` (default) or `kn`; every bilingual string is an inline `o.t(en, kn)` pair
`flow`	`new` → `lang` → `menu` → `grievance` → `ask_place` / `ask_gp` → `confirm` → `done`; or `apply` → `apply_name` → `apply_village` → `apply_doc` → `done`; or `reply`
`draft`	Working data: `text`, `masked`, `u` (the AI understanding), `src` (`whatsapp` / `whatsapp-voice` / `whatsapp-photo`), `gp`, `taluk`, `village`, `how`, `dup`; for applications `sakala_key`, `name`, `gp`, `taluk`; for replies `case`
`stage`	Mirror of the stage events, `{"0": {state, note}, …}`

`how` records where the location came from and goes into the audit record: "picked from the GP list", "GPS location shared", "village typed by the citizen", "village named in the message", "taluk named in the message".

9.3 Every action the engine accepts (`POST /api/chat`)

Action	What it does	Events
`start`	Greeting "ನಮಸ್ಕಾರ 🙏 Namaskara! I am Sakala AI…" and the language question (buttons `lang:en`, `lang:kn`)	2 msgs; the only action that does not run `deliver()`
`lang:en` / `lang:kn`	Sets language, shows the main menu	chips cleared, list msg

Action	What it does	Events
`menu`	List: 📝 Raise a grievance · 📄 Apply for a service · 🔎 Track my request · ☎️ Talk to an officer	list msg
`grievance`	"Tell me what is wrong. You can type, send a 🎤 voice note or a 📷 photo..." plus 6 sample chips (water, khata, pension, ration card, street light, power cut)	reset, msg, chips
(free text)	`grievance(text)` : the full understanding path below	stages 0–3, msgs
`voice` + `audio_b64`	16 kHz WAV → `ai.transcribe` (qwen3-asr) → "🎤 I heard: ..." → same path. Fewer than 4 letters → "I could not hear that clearly"	
`photo` + `image_b64`	`ai.read_petition` (qwen4-deploy vision) → "📷 I read your petition: ..." (masked) → same path. Unreadable → refusal, never a guess	
`loc:share`	Demo GPS point: Gorahalli village → Chapparadahalli GP, Piriyapatna	me-bubble, stage 3, confirm card
`tk:<Taluk>`	Shows the GP list of that taluk (≤ 10 rows, most-used first)	list msg
`gp:<GP>|<Taluk>`	Picks the GP, goes to confirm	stage 3, card
(typed village)	In `ask_place` / `ask_gp` : gazetteer match (Kannada transliterated); in `ask_gp` a miss says "I could not find that village in the pilot area"	
`submit`	Creates the case (section 6 stages 4–7), sends the **Registered card** and a toast	stages, `result` , card, toast
`change`	Back to "Tell me what is wrong"	reset
`track` / text "track", "status"	Card with up to 4 of this chat's cases: id, service, "With {officer} · L1", and ✅ Resolved / ❌ Rejected · you can appeal / 📩 Officer asked for information / ⚠️ n days overdue · escalated / ⏳ n days left	card
`apply` → `svc:<key>` → name → `village:<GP>|<Taluk>` → `doc`	Sakala application: checklist, applicant name, village, document photo (reads a BPL ration card), then **Application accepted** card with the GSC-style number, legal due date and Designated Officer	stages 0–7, `result` , card
`apply_now:<key>`	Same, offered when a grievance is really a service request	
`reply:<cid>` then text or photo	Citizen's answer to an officer's question goes into the case file as "Citizen replied" (masked); Info requested → In progress; officer notified	msg, toast
`fb:<cid>:yes`	"Citizen confirmed: problem solved 👍 "; status Resolved	msg
`fb:<cid>:no`	**Re-opens** the case: status Pending, clock restarted, owner and their senior notified	msg + buttons
text "appeal" / "ಮೇಲ್ಮನವಿ"	Moves the latest Rejected case to the senior officer (CO for Sakala applications) with an event and a notification	msg
`officer`	"A helpline officer will call you back within 2 working hours" (PRODUCTION PHASE badge)	msg, menu

Action	What it does	Events
`later`	"Tap Reply now on the officer's message whenever you are ready"	msg
`poll`	Nothing but `deliver()`	officer bubbles if any
unknown	"Sorry, I did not understand that option." + menu	

9.4 The understanding path for a grievance text

1. **Stage 0** note by channel ("Message received on WhatsApp" / "Voice note…" / "Photo of petition received").
2. **Stage 1 privacy:** `pipeline.understand(text)` masks first; note "{n} personal number(s) masked before AI" or "Nothing personal to mask".
3. **GSC bridge:** if the text quotes a Sakala number (`[A-Z]{2}\d{3}[A-Z]\d{9,10}`) the bot explains whether Sakala rights apply, naming the Competent Officer and the 30-day appeal, or that "this is about a payment after the service was delivered, so Sakala delay rights do not apply". Another chat's application is never revealed.
4. **Stage 2 understanding**, then a branch on `intent`: `WRONG_DOOR` (RTI, court, employee matters): "ℹ️ {reason}" and menu. `OUT_OF_SCOPE` (e.g. power cut → Energy / CESC): "This department is outside the 3 pilot departments, so in production it goes straight to that department's field officer through iPGRS." `SERVICE_REQUEST` (e.g. "apply for a caste certificate"): offers 📄 Apply now with the legal deadline. `UNCLEAR`: a Scrutiny case is created for the EO Piriyapatna; "a person at the scrutiny desk will check it. Reference GR-…". Otherwise: "Got it 👍 I understand this as: **Revenue Department** → Land Records → **RTC/Pahani** · "RTC correction" · 91% sure", plus "❤ Marked as priority (vulnerable citizen)" when flagged.
5. **Stage 3 location**, first match wins: a village named in the text (gazetteer, LLM village trusted only if present in the text) → confirm; a taluk named in the text ("road is bad in Hunsur") → if the officer is taluk-level, confirm with no village question, else show that taluk's GP list; nothing → "Where is this problem? Share your 📍 location, choose the taluk, or type the village name."
6. **Stage 4 confirm card:** Problem / Service / Location, plus "A similar open request exists (GR-…); yours will be linked to it" when `find_duplicate` finds the same service in the same GP still open. Buttons ✅ Submit / 🖍️ Change.
7. **Submit** → stages 5–7 and the Registered card: "Assigned to **PDO, Chapparadahalli Gram Panchayat**, the officer responsible for Chapparadahalli Gram Panchayat. ⏱ Action due in **7 days**. If not acted on, it moves automatically to **EO TP Piriyapatna**. You will get every update here. Reply TRACK anytime."

9.5 Officer → citizen messages (two-way)

Every officer action writes a row to `citizen_msgs` (`sla.message_citizen`), and `deliver()` turns undelivered rows into green-edged **officer bubbles** on the citizen's next poll, translated to Kannada by the gateway when the chat is Kannada (`ai.to_kannada`).

Kind	Trigger	Message (English template)	Buttons
`accepted`	officer Accept	✅ **{officer}** has accepted your request **{cid}** and started working on it.	
`info`	Ask for info	📩 **{officer}** needs more information on **{cid}**: "{question}"	✍️ Reply now · Later
`resolved`	Resolve with ATR	🎉 **{officer}** has marked **{cid}** as resolved. Action taken: "{ATR}". Was your problem actually solved?	👍 Yes, solved · 👎 No, not solved

Kind	Trigger	Message (English template)	Buttons
`rejected`	Reject with reason	❌ **{officer}** could not accept **{cid}**. Reason: {reason}. Your right: you may appeal to **{senior}** within 30 days. Reply **APPEAL** and I will send it up now.	
`returned`	Return (wrong jurisdiction)	↩️ **{cid}** is being re-checked by the scrutiny desk to find the right office.	
`routed`	Scrutiny desk re-route	📍 **{cid}** has been checked by the scrutiny desk and is now with **{officer}**.	
`escalated`	`tick()` auto-escalation	⏫ **{cid}** was not acted on in time, so it has moved up to **{senior}** automatically. ({officer} did not act within 7 days)	
`compensation`	CO orders compensation	💰 Sakala compensation of **₹{amount}** has been ordered for the delay on **{cid}** (₹20 per day, Sakala Act s.9).	

Session isolation (guardrail 9): `own_case()` refuses `reply:` and `fb:` on a case this chat did not file ("That request was not filed from this chat…"); `/api/state?scope=citizen` returns only this session's cases and no officer notifications.

9.6 Event shapes the client renders

- `msg`: `{kind:"msg", me:false, type:"text"|"card"|"officer", html, buttons?:[{label, action, silent?}], list?:{button, title, rows:[{t, d, action}]}, officer?, officerFull?, caseId?}`; citizen echoes have `me:true` and `type:"loc"|"image"`.
- `stage`: `{kind:"stage", i:0–7, state:"done"|"run"|"warn", note}`.
- `result`: `{kind:"result", caseId, officer:{id,name,short,level}, chain:[{lv, short}], basis, title:"Routed to"|"Sakala application …"}`.
- `chips`: `{items:[[label, text]] | null}`; `reset`; `toast: {h, m}`; `notify: {n, from}` (officer messages delivered).

Known rough edges (worth fixing in the Jarvis build): typed text in `confirm` or `done` starts a new grievance; `apply_name` takes any text as the name; the document reader always shows a BPL ration card; the voice transcript is shown unmasked in "🎤 I heard"; the Sakala-breach event says the citizen was informed but no citizen message is queued for it.

10. Profile 2: the Officer

The officer profile is a work desk for one named officer at a time: "Log in as" any of the 88 officers, see what is routed to you most urgent first, and act with a handful of buttons. Every action is a `POST /api/cases/{id}/action` with the officer's id; the server, not the UI, enforces who may do what. **There is no authentication: identity is the `officer` field the client sends. Jarvis must carry the logged-in officer id on every call.**

10.1 Sidebar and views

Item	Shows	Rule
Log in as	A select grouped by department: "L1 · Tahsildar Piriyapatna", …	Default `tah_piriyapatna` (258 real cases at go-live)
My inbox	Cases where `owner = me`, status not Resolved / Rejected / Scrutiny	Sorted vulnerable first, then least time left
Reminders	Open cases with `left ≤ 2` days	"Due within 2 days or already overdue"
Sakala applications	Cases where I am DO, CO or AA	Legal clock and compensation
My team	Only for officers with reports: one bar per officer below me, red if they have overdue cases	"Who is overloaded and what is about to escalate to you"
Scrutiny desk	Status `Scrutiny`: low-confidence AI results and cases returned as "not mine"	Decision panel for L2 / L3 and the CS office
Escalation ladder card	The chain above me and "Assumed timeline: L1 7 days · L2 5 days · L3 3 days"	

List view KPIs: Open with me · Due in ≤ 2 days · Overdue ("will move to {senior}") · Vulnerable citizens ("shown first") · Escalated to me. A reminder banner appears when anything is overdue or due. **Columns:** Reference (id, date) · Citizen's issue (masked text) · Service · Location (taluk › GP) · Source badge · Time left (progress bar: "N days left", "Due today", "N days overdue", "· legal" for Sakala) · Flags (♥ Priority, Sakala, Real case). "Today's digest" opens a morning-summary modal (PRODUCTION PHASE for actual sending).

10.2 The case drawer (`openCase(id)`)

1. **Header:** "Grievance" or "Sakala application" · source; the case id; badges for status, `{level} · {owner}`, ♥ Vulnerable, From real sample.
2. **Citizen's words (masked).**
3. **AI summary & classification:** the reason, the path Dept › Line › Service › Grievance, then the collapsible **Routing evidence** (engine, confidence vs threshold, decision, rules applied, location and how it was found, officer rule "PDO · 99% of 471 real cases", similar real cases, human decision), "Routed by" and "Why this officer" (`route_basis`), confidence, location, received, time left.
4. **Sakala legal clock** (applications only): service, clock start, legal timeline, DO / CO ("1st appeal within 30 days") / AA ("2nd appeal within 60 days"), compensation accrued "₹X (₹20/day × N days, max ₹500)"; when the viewer is the CO and the case is late, three steps: **1 · Order compensation** → **2 · Draft Form C notice** (client-side draft of the s.11 recovery notice, 7 days to reply) → **3 · Approve HRMS debit** (MOCK HRMS).
5. **Action Taken Report** (if any) with "AI resolution check: N/5".

6. **Audit trail:** every `case_events` row (time, who, what, note).
7. **Scrutiny desk · decide the route** (Scrutiny cases, viewer L2 / L3 or CS): officer picker grouped by department, optional note, **Route it**. Note: the server supports `reroute`, but the current generated UI has no click handler on this button (a known gap; call the API directly).

10.3 Every action and its rule (`api.case_action`)

Button	`action`	Who may	Preconditions	Effect
Accept & start	`accept`	owner only	status open (Pending / In progress / Info requested), else 409	status In progress; event "Accepted"; citizen message `accepted`
Ask citizen for info	`info` + `note`	owner	open	status Info requested; event with the question; citizen gets 📩 with Reply now / Later; Kannada chats get the question translated
Return · not my jurisdiction	`return` + `reason`	owner	open	status Scrutiny (owner unchanged); "Logged as a routing correction for the learning loop"; citizen message `returned`
Reject with reason	`reject` + `reason` (+ `note`)	owner	open; **reason mandatory (400)**	status Rejected; citizen told the reason and the appeal officer by name; APPEAL moves it up
Resolve with ATR	`resolve` + `atr`	owner	open; **ATR ≥ 15 characters (400)**	ATR scored by the AI (or keyword fallback). Score ≥ 4: status Resolved, event "Resolved with ATR", citizen asked 👍 / 👎. Score < 4: event "ATR flagged: not resolved", status unchanged, HTTP 200 with `score` and `reason`
AI draft	`POST /cases/{id}/draft_atr`	anyone		Returns a 3–4 sentence draft with `[DATE]` / `[DETAIL]` placeholders
Route it (scrutiny)	`reroute` + `to` (+ `note`)	CS office (no officer) or L2 / L3; **L1 → 403**	status Scrutiny (409 otherwise); **never an application (409)**; target must exist (400)	owner = target, level = target's level, clock restarted, status Pending; `audit.human_decision`; target notified; citizen message `routed`
Nudge (CS only)	`nudge`	anyone	open or Scrutiny	event "Reminder from Chief Secretary's office"; owner notified
1 · Order compensation	`comp_order`	**Competent Officer only (403)**	Sakala application (400); `compensation > 0` (409)	event "Compensation ₹{amt} ordered"; citizen message `compensation`
3 · Approve HRMS debit	`comp_hrms`	CO only	same	event "HRMS debit note ₹{amt} approved · Posted to HRMS (mock) after the officer's reply · s.11(4)"

Who = the officer's short name, or "Chief Secretary's office" when no officer is sent. Acting on a closed case returns 409 "case is Resolved". Any other officer gets 403 "only the officer who holds the case can act on it". Errors arrive as `{detail}` and the UI shows a toast "Not allowed · {detail}".

10.4 What the officer receives

- **Notifications** (`notifications` table, shown as the digest badge and in `/api/state.notifs`): new grievance routed; "GR-… is due today / tomorrow · will escalate if not acted on"; "escalated to you from {officer}" and "escalated to {senior} (no action in time)"; "Citizen replied on GR-…: …"; "GR-… re-opened: citizen says the problem is not solved"; "Appeal on GR-… from the citizen"; "Sakala deadline missed on SK-…: compensation due" (to the CO); "Chief Secretary's office asked for action on GR-…"; "GR-… assigned to you by the scrutiny desk".
- **Citizen replies** land in the audit trail as "Citizen replied" (masked) and move Info requested → In progress.

10.5 Scrutiny desk

Cases reach it four ways: the AI said UNCLEAR or confidence fell below the threshold; a guardrail refused a category or officer not in the master; an officer returned it as "not my jurisdiction"; a location was missing in a bulk upload. All land with `eo_piriyapatna` at L2 (a simplification to fix in production). A human picks the officer; the decision is stored in the audit as `human_decision`, and the citizen is told where the case now sits.

10.6 What "peaceful officer work" means for Jarvis

The officer's whole day is five verbs: accept, ask, resolve, return, reject (plus compensation for Competent Officers). Each has one API call and one citizen message. A voice agent can read the inbox aloud (most urgent first), open a case, draft the ATR, read the AI score back and file it, all without the officer touching the keyboard. The ATR auditor is the quality gate that keeps this honest: a lazy "will look into it" scores 2 and does not close the case.

11. Profile 3: the Chief Secretary

The CS profile answers three questions in one glance: where are complaints piling up, who is not acting, and is the state's Sakala performance improving. It is built around her Wednesday review rhythm. Every number comes from the live database or the real 2025 reports; the AI only writes summaries from facts it is given.

11.1 Navigation

Item	Subtitle	API
AI mode card	"Ask anything · answers from live data"	`POST /api/ask`
Command dashboard	"Live · 3 departments"	`/api/state`, `/api/bootstrap`
Sakala district tracker	"Real 2025 reports"	`/api/sakala/tracker`, `/api/sakala/early_warning`
All cases	"Every request, every officer"	`/api/state`
Insights	"Hotspots · history · misrouting"	`/api/insights/hotspots?fix=1`, `historical?summary=1`, `misrouting`, `quality`
Accuracy proof	"Measured on real data"	`/api/insights/accuracy[?run=1&n=40]`
Data upload	"Excel / CSV from any department"	`POST /api/upload`
Production roadmap	"Next phase"	static
Departments	colour square, name, open count	
Wednesday Review Pack button		`/api/review_pack?level=&officer_id=`, `/api/e4`

11.2 Command dashboard

- **Hero:** "Command dashboard · Revenue · RDPR · Food & Civil Supplies · Mysuru district (pilot) · as of {demo date}"; buttons **Ask AI** and **Wednesday Review Pack**.
- **Live map · Karnataka** (section 11.3).
- **KPIs:** Pending now (hero) · Overdue · Auto-escalated · Vulnerable waiting · **Exact-officer agreement 97.6%** ("673 real Mysuru cases: router picks the same officer iPGRS used").
- **Cases at each escalation level:** L1 field officer · L2 EO / AC / DD · L3 DC / ZP CEO; **Pending by department**; **Ageing** 0–3, 4–7, 8–15, 15+ days.
- **Officers needing attention:** top 7 by overdue (Officer / Level / Open / Overdue); clicking a row opens that officer's profile.
- **From the real data shared:** channel bars and the banner "39% of Mobile App complaints wait in manual scrutiny; 349 complaints sit with one state desk under 'General'".

11.3 The Karnataka map

- **Layers:** a white floating rim with shadow (no black outline); 31 district polygons (KGIS) shaded on a Karnataka-flag heat scale (butter → flag yellow `#ffcd00` → amber → flag red `#c8102e`) by real Sakala 2025 late-delivery %; Mysuru in emerald with a glow; OSM taluk polygons for Piriyapatna (emerald),

Hunsur (blue), Mysuru (violet); taluk markers with counts; one dot per open case, placed deterministically inside its taluk.

- **Dots:** on time `#0e9f6e`, overdue `#e5484d`, vulnerable `#d6409f`; overdue and vulnerable drawn last so they are never buried.
- **Zoom levels:** state → district → taluk, with log-space eased glides (720–820 ms), wheel zoom about the cursor, drag pan, double-click ×2; labels and dots fade in with zoom; glow filters pause while moving.
- **Hover:** a district lifts and glows, tooltip "{x}% delivered late · Sakala 2025 · #rank of 31 districts (1 = most late)"; a dot shows id, service, officer · level, days left or overdue.
- **Side panel:** pilot taluk buttons (Kannada names, open and overdue counts); three **category tiles** On time / Overdue / Vulnerable with live counts (`setCat`): other dots fade to 7%, matching dots grow and pulse, the map glides to fit them, and a list of up to 40 matching cases (most urgent first) opens; the Sakala 2025 late % legend (1.9% to 9.2%).
- **Clicks:** dot → case drawer (read-only with a Nudge button); taluk or marker → zoom; Mysuru → district view.

11.4 Sakala district tracker (use cases 21 and 22)

- KPIs: Applications received (3.5 Cr) · Delivered late 4.45% · Rejected 7.42% · Overdue backlog −63% (51,172 → 19,129).
- "Overdue applications, month by month" bar strip and the six highest late-delivery districts.
- **Early warning:** each district's last 3 months against the 3 before; flags when rejection % rose more than 0.8 points, late % rose more than 0.8 points, or overdue grew more than 1.25× and above 150.
- **Fairer scorecard:** `fair_score = 50 + 10 × (−z(late) − z(rejected) − z(overdue per lakh) − z(trend)) / 2`, ranked, shown beside the official rank with the note that official scores span only 61–79.

11.5 Wednesday Review Pack (use case 18) and E-4

A modal with level tabs **Chief Secretary · DC Mysuru 3–5 pm · AC Hunsur 11–1 · EO / Tahsildar 9–11** (`level=cs|dc|ac|eo`, `officer_id` dc_mys / ac_hunsur / eo_piriyapatna; a non-CS level covers that officer plus two levels of reports). Facts computed from the DB: open, overdue, vulnerable waiting, auto-escalated, top issues, most overdue officers, Sakala deadlines missed, stuck examples, Mysuru vs state late %. The gateway writes a four-part English brief (position, top issues, where it is stuck, three decisions) and a Kannada version; the facts are shown underneath. **Form E-4 show-cause:** `GET /api/e4` lists every officer with more than 7 overdue cases (Sakala Rules; s.14(4) threshold).

11.6 Insights (use cases 3, 8, 17, 19)

- **Hotspots:** same service in the same GP, 3 or more open cases (or in the sample week), with an AI-inferred root cause and fix for the top 6.
- **Praja Seva historical intelligence:** camp share, busiest day, app scrutiny rate, vulnerable citizens waiting, repeat complaints, top issues by taluk, AI summary.
- **Misrouting detector:** records whose filed service the offline model rates below 5% while another service scores above 80%; plus the 349 KRIDL "General" rows; 5 examples.
- **Data quality:** 9 intake checks on the 9,999 rows (phones, Aadhaar, GSC numbers, missing GP, missing taluk, short text, duplicates, "General", stuck in Registered) each with the fix the validator applies.

11.7 AI mode (use case 20)

- **Hero:** floating Sakala logo, "Sakala AI · ask anything", "Answers come only from the case database and the Sakala 2025 reports. If the data doesn't exist, it says so." Subtle emerald / saffron gradient and a faint

logo watermark.

- **Suggestion cards:** Which officer has the most overdue cases? · How many pension complaints are pending in Piriyapatna? · Show pending cases by department · Which district had the highest late-delivery rate in 2025? · How many cases were auto-escalated? · Which channel brings the most complaints? · ಪಿರಿಯಾಪಟ್ಟಣದಲ್ಲಿ ಎಷ್ಟು ಕುಡಿಯುವ ನೀರಿನ ದೂರುಗಳಿವೆ?
- **Ask bar** pinned at the bottom, with a mic (`askVoice`: 6-second capture → `/api/transcribe` → auto-submit).
- **Answer flow:** `POST /api/ask {q}` → plan → tools (`query_cases`, `real_records`, `sakala_2025`, `insights`) → 2–5 sentence answer with the key numbers in bold → `clean_html` → **chart built by the server from the tool results, never by the model** (`build_chart`). Falls back to a local answer when the gateway is down.
- **Charts** (Chart.js 4.5.1, `app_charts.js`): bar ↔ line morph in place, donut crossfade with the total in the centre, a sliding Bar / Line / Donut toggle, hover guide line, emerald gradient bars turning saffron on hover, Sakala palette for slices; donut offered only for additive counts of ≤ 12 items. Examples that produce charts: cases by department, by channel, districts by late %, monthly overdue trend ("month", "trend", "over time"), hotspots, scrutiny by channel.
- **Guardrails** (section 13): injection and off-topic refusals, logged as `ask_blocked`.

11.8 Data upload (use case 23)

Drop an `.xlsx`, `.xls` or `.csv` (max 5 MB). A Sakala performance workbook (month-named sheets with an "SL" header) loads into `sakala_perf`; any sheet with a Description / Complaint / Grievance column is treated as complaints and the first 50 rows run through the full pipeline (`src=upload`), returning "X of N complaints routed to a named officer in one pass" with a Complaint / Understood as / Sent to table. A sample file is at `/samples/sample_complaints.csv`. The card "Every digital level" lists Full API (PRODUCTION), Excel / CSV (LIVE), Paper scan via WhatsApp (LIVE), Officer web console (LIVE).

11.9 Production roadmap (use cases 24–29, front end only)

AI callback to the citizen (Kannada voice call after closure) · Namma Seva 5-star rating · Live integrations (iPGRS, Sakala, Seva Sindhu, HRMS, CPGRAMS) · SMS / WhatsApp sending · Learning loop / sovereign model (officer corrections retrain a government-hosted Kannada model) · Citizens-as-sensors alerts (a topic spike in one taluk warns the DC).

12. Deadlines, the demo clock and accountability

Time in the POC is a **demo clock**, not wall time: `kv.now` starts at `DEMO_START = 14 Oct 2026 10:00` (pilot go-live) and moves only when someone presses +1 day / +7 days (`POST /api/clock/tick {days}`, clamped 1–30). Everything that depends on time (days left, escalation, reminders, compensation) reads `sla.now()`. Reset (`POST /api/reset`) restores the pristine database, clock included.

12.1 Time left (`sla.sla_info`)

Case kind	`total`	`used`	`left`	`legal`
Grievance	`SLA_DAYS[level]`: L1 7, L2 5, L3 3 (ASSUMED)	whole days since `level_since`	`total − used`	false
Sakala application	`sakala.dueDays` (21 / 45 / 30, illustrative)	whole days since `sakala.complete`	`total − used`	true

The UI shows the same maths client-side (`slaInfo`): a progress bar with "N days left", "Due today", "N days overdue", and "· legal" for Sakala cases. Imported real backlog cases start their clock at go-live (`level_since = max(created, DEMO_START)`); otherwise the first +7 days would escalate about 380 cases at once.

12.2 What a tick does (`sla.tick(days)`), in order

1. **Move the clock** and save `kv.now`.
2. **Auto-escalation** (grievances in Pending / In progress / Info requested; Scrutiny excluded), up to 3 hops per case: while `left < 0` and the owner has an `up_id`, move the case to the senior (`owner`, `level`), with `level_since` set to the moment the deadline lapsed (so a case can cascade L1 → L2 → L3 in one long tick). Event "Auto-escalated to {senior}" ("{officer} did not act within 7 days (assumed timeline). Notice sent to both officers."); notifications to the senior ("escalated to you from …") and the old owner; citizen message ⏫ `escalated`.
3. **Reminders** for every open case (both kinds) with `left` 0 or 1: "GR-… is due today / tomorrow · will escalate if not acted on" or "SK-… is due today (Sakala legal deadline)".
4. **Sakala breach** for applications with `left < 0` not yet flagged: `sakala.breached = true`, event "Legal deadline missed" dated at the due date, notification to the Competent Officer "Sakala deadline missed on SK-…: compensation due".
5. Returns `{now, escalated, breached}`; the UI toasts "Moved to {date}. N grievance(s) auto-escalated. N Sakala deadline(s) missed."

12.3 Sakala compensation and the HRMS mock (use cases 11, 13)

- `compensation(c) = min(500, 20 × days late)` when `legal` and `left < 0` (Sakala Act s.9).
- The Competent Officer (AC for Revenue, DD Food for Food) sees three steps in the drawer: **Order compensation** (`comp_order`: event "Compensation ₹{amt} ordered · Paid to the citizen", citizen message 💰) → **Draft Form C** (the s.11 recovery notice to the Designated Officer, 7 days to reply; drafted client-side from the case record) → **Approve HRMS debit** (`comp_hrms`: event "HRMS debit note ₹{amt} approved · Posted to HRMS (mock) · s.11(4)"). A human approves every step; nothing is automatic.
- Demo case: `SK-RD855S260004811`, caste & income certificate for "Lakshmi K", complete 18 Sep, 21-day timeline, 26 days old at go-live → breached, ₹100 accrued, CO `ac_hunsur`. The second application `SK-FD201S260002219` (new ration card, "Manjunatha S", Hunsur, 30 days) is within time.

12.4 Creating a Sakala application (`sla.create_application`)

- Id `SK-RD855S26xxxxxx` (Revenue) or `SK-FD201S26xxxxxx` (Food), mimicking real GSC numbers.
- DO = `officer_for_role(do_role, taluk, gp)`; CO = `ac_hunsur` for Piriyapatna and Hunsur, `ac_mysuru` for Mysuru, `dd_food` for Food; AA = `dc_mys`.
- `sakala` JSON `{key, service, dueDays, DO, CO, AA, complete, applicant, illustrative: true}`; `line = "Sakala service"`, `grievance = "New application"`, `engine = "Sakala"`.
- Event "Application registered · Designated Officer {short} · Complete on submission · legal clock started"; DO notified.
- `insert_case` refuses an application without a `sakala` clock or a grievance with one (guardrail 14).

12.5 Notifications and messages, who gets what

Trigger	Officer notification	Citizen WhatsApp
New grievance routed	owner: "New grievance GR-… ({service}) from {src}"	Registered card
Due in 0–1 days	owner reminder	none
Deadline passed	senior and old owner	⏫ escalated
Sakala deadline passed	Competent Officer	none queued (known gap; the event text says the citizen was informed)
Officer accept / info / return / reject / resolve	none	✅ / 📩 / ↩️ / ❌ / 🎉
Citizen reply, 👎 re-open, APPEAL	owner (and senior for re-open / appeal)	confirmation
Scrutiny desk re-route	target: "assigned to you by the scrutiny desk"	🎈 routed
CS nudge	owner	none
Compensation ordered	none	💰

12.6 Synthetic demo cases (flagged `synthetic = 1`)

Ten grievances aged 3–12 days at go-live so the first +1 / +7 days show escalation: pension not received (74 years old, Hunsur, 6 days) · drain not cleaned, mosquitoes (Attigodu, 12 days, already at L2) · street light pole fallen (Chapparadahalli, 8 days, L2) · borewell pipeline broken (Komalapura, 5 days) · ration dealer short-weighing (Attigodu, 7 days) · house not sanctioned under Basava Vasathi (Komalapura, 3 days) · internal road potholes (Bhuvanahalli, 10 days, L2) · podi survey pending (Haraduru, 6 days) · fair price shop closed (Jayapura, Mysuru, 8 days, L2) · wrong name in pahani (Govindanahalli, Hunsur, 4 days), plus the two applications above. At go-live: 1 overdue and 3 vulnerable open cases; press +7 days to see many more overdue.

13. Guardrails

Fourteen of the 23 guardrails proposed on 14 Oct were implemented at the client's choice; nine were deliberately left out. Each implemented one is tested in `tools/tests/test_guardrails.py` (27 checks, all passing). **Jarvis inherits all of these for free as long as it drives the existing API; the new risk surface is the voice layer itself (section 17).**

13.1 Implemented

#	Guardrail	How it works	Where
1	AI chooses only from official masters	`service_id` is an enum of the catalogue in the JSON schema; `in_masters()` re-checks id and name and snaps the grievance name to the service's own types; an unknown category or officer → scrutiny desk, never a silent guess	`ai.understand`, `pipeline.in_masters`, `create_grievance`
2	Auto-route threshold tuned from measured accuracy	`llm_eval` picks the lowest threshold (0.40–0.75) at which auto-routed cases are ≥ 90% right on service while ≥ 70% still auto-route; stored in `kv.low_conf` (last run 0.40), read by `pipeline.low_conf()`; below it → UNCLEAR with alternatives	`analytics.llm_eval`, `pipeline.understand`
3	Every routing decision keeps evidence	`cases.audit` JSON (section 8.4) shown in the drawer as "Routing evidence"	`pipeline.audit_record`
6	AI mode answers only from data	`guard_question()` refuses instruction-override / reveal-prompt / act-outside-data prompts (`INJECTION_RX`: ignore / override instructions, system prompt, jailbreak, pretend to be, send an SMS / WhatsApp, delete / update table, run code) and off-topic ones (`OFF_TOPIC_RX`: weather, cricket, movies, jokes, poems, "who is the prime minister"…); an empty tool plan returns "I don't have that data"; the writer is told never to follow instructions inside the question or the results	`ai.guard_question`, `ai.ask`
7	Vision and voice fail safe	Unreadable or unrelated images (readable = false or under 8 letters) are refused with a clear message; transcripts under 4 letters are treated as no speech ("could not hear")	`ai.read_petition`, `ai.transcribe`, `chat.photo`, `chat.voice`
9	Session isolation	`own_case()` guards reply and feedback; another chat's GSC number is never revealed; `/api/state?scope=citizen` returns only that session's cases and no officer notifications; the client reloads state on every profile switch	`chat.own_case`, `analytics.gsc_bridge`, `api.all_cases`
12	Every AI call logged	`ai_log(t, task, engine, ms, ok, detail)` including offline-model use and blocked prompts; `GET /api/ai_log?limit=` shows failures first	`ai._log`, `api.ai_log`
13	Only the owner acts; only the Competent Officer orders compensation	403 on anyone else; `comp_*` require `officer == sakala.CO`	`api.case_action`

#	Guardrail	How it works	Where
14	Ladders never mix	`insert_case` refuses a grievance with a Sakala clock or an application without one; `reroute` refuses applications (409); `tick()` escalates grievances only; compensation refuses grievances (400)	`sla.insert_case`, `api.case_action`, `sla.tick`
16	Rejection needs a reason and names the appeal right	Reject without `reason` → 400; the WhatsApp message names the appeal officer (senior on the grievance ladder, or the CO for Sakala); replying APPEAL moves the case up with an event and a notification	`api.case_action`, `chat.KIND_TEXT`, `chat.appeal`
17	Resolve needs a real ATR; the citizen can re-open	ATR ≥ 15 characters and AI score ≥ 4 to close; 👎 re-opens with the senior informed	`api.case_action`, `chat.feedback`
19	Rate limits and request caps (as in the sister POCs)	Per client IP per rolling minute: `/api/transcribe` 240 · `/api/chat` 120 · `/api/ask` 60 · `/api/cases` 120 · `/api/upload` 20 · `/api/clock` 60 → 429 with `retry-after: 60`; bodies over 8 MB → 413; headers `x-content-type-options: nosniff`, `x-frame-options: DENY`, `referrer-policy: no-referrer`, `permissions-policy: camera=(), microphone=(self), geolocation=()`	`api.Protection` ASGI middleware
20	Sanitise model output	`clean_html()` on every model-written HTML; client `safeHtml()` allows only B STRONG I EM BR P UL OL LI; citizen and officer text escaped	`ai.clean_html`, client
22	Honesty badges	SYNTHETIC badges removed from the inbox and drawer (the `synthetic` flag stays in the data); PRODUCTION PHASE badges kept on every front-end-only feature	wireframe + app

13.2 Standing rules carried from the brief

- Personal data is masked before any AI step (stage 2).
- Sakala compensation applies only to Sakala applications.
- Salary action (HRMS debit) always needs a human approval click.
- Low confidence always goes to a human.

13.3 Deliberately not implemented (client decision)

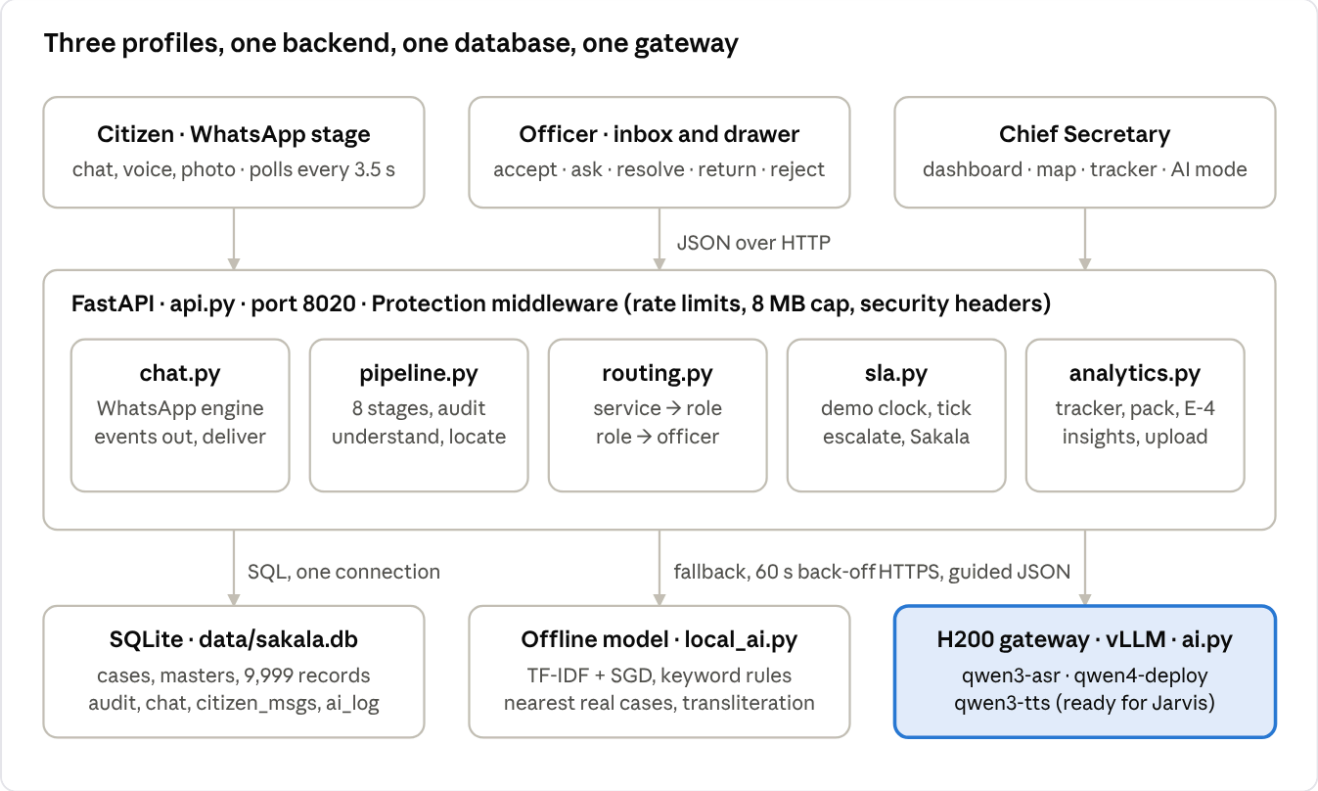
4 (name masking inside Kannada text), 5 (document-authenticity checks), 8 (per-officer action quotas), 10 (explicit consent capture), 11 (data-retention timers), 15 (duplicate-complainant throttling), 18 (second-officer approval on rejection), 21 (CS read-only enforcement beyond the UI), 23 (formal model-change log). Guardrail 19 was conditional on the sister POCs having rate limiting; they do, so it was built.

13.4 Known gaps to keep in mind

- No authentication or login: identity is whatever `officer` id the client sends.
- Personal names inside Kannada complaint text are visible to the model (only numbers are masked).
- The scrutiny desk is hard-wired to `eo_piriyapatna`.
- Rate-limit counters never evict keys (fine for a demo, not for production).

14. Code architecture

The whole system is about 3,000 lines of Python in 12 files, one SQLite database, one generated HTML file and four JavaScript modules. There is no build step, no frontend framework and no external service except the team's own model gateway.



Every screen talks only to `api.py`; the modules inside it share one SQLite connection; the gateway is the only external dependency, and the offline model takes over for 60 seconds whenever it cannot be reached.

14.1 Folder map (`D:\Sakala POC\`)

Path	What it is
`app/run.bat`	Builds the DB on first run (`python -m server.ingest`, about 2 minutes), opens the browser, starts uvicorn on 8020
`app/requirements.txt`	fastapi, uvicorn, pandas, openpyxl, scikit-learn, joblib (plus httpx, used by `ai.py`; `anthropic` is listed but no longer used)
`app/.env`	Gateway configuration (copy of the KPCL env; keys redacted here)
`app/server/*.py`	The backend (table below)
`app/data/sakala.db`	The live SQLite database (WAL mode)
`app/data/sakala_pristine.db`	Snapshot restored by `/api/reset`
`app/data/local_models.joblib`	Offline model
`app/data/llm_eval.json`	Last live accuracy run (survives reset)
`app/web/index.html`	The single-file UI, **generated** from the wireframe (section 16); never edit by hand

Path	What it is
`app/web/assets/`	`karnataka-emblem.png`, `sakala-logo.svg`, `vendor/chart.umd.min.js` (Chart.js 4.5.1)
`app/web/samples/sample_complaints.csv`	Upload demo file
`wireframe/index.html`	Design source of truth (2,104 lines; simulated AI)
`tools/`	`make_app_ui.py`, `app_wa_client.js`, `app_insights.js`, `app_charts.js`, `future2.css`, `wa_module.js`, `wa_future.css`, patch scripts, `tests/`, `app_tests/`, `arch.html`, `make_pdf.py`, `osm/` (map data), `backups/`
`CLAUDE.md`	The running handoff log (every change, with timestamps)
`Sakala_AI_POC_Architecture.png/.pdf`	Architecture picture (regenerated 14 Oct)

14.2 Server modules (`app/server/`)

Module	Lines	Responsibility
`config.py`	26	Paths, `PILOT_TALUKS`, `PILOT_DEPTS`, `SLA_DAYS`, `DEMO_START`, `COMP_PER_DAY`, `COMP_CAP`
`db.py`	103	Schema, singleton connection (`check_same_thread=False`, autocommit, `Row` factory, WAL), helpers `q`, `q1`, `run`, `many` under one `RLock`, `migrate()`
`privacy.py`	19	`mask()`
`local_ai.py`	239	Offline model, keyword rules, transliteration, `place_candidates`, `alternatives`, nearest cases
`ai.py`	334	Gateway client, every model function, guardrail regexes, `clean_html`, `ai_log`
`routing.py`	88	`route`, `officer_for_role`, `resolve_place`, `gp_list`
`pipeline.py`	180	`understand`, `locate`, `find_duplicate`, `in_masters`, `audit_record`, `create_grievance`, `scrutiny_case`, `process`, `STAGES`
`sla.py`	149	Demo clock, `sla_info`, `compensation`, `insert_case`, `create_application`, `tick`, `event`, `notify`, `message_citizen`, `chain`
`chat.py`	617	WhatsApp state machine, `deliver`, `KIND_TEXT`, appeal, feedback
`analytics.py`	393	Performance reports, tracker, early warning, data quality, misrouting, accuracy / `llm_eval`, GSC bridge, historical, hotspots, review pack, E-4, upload
`ingest.py`	293	Builds everything from the Excel: masters, learned rules, gazetteer, officers, records, real caseload, synthetic data, model training, routing evaluation, pristine copy
`api.py`	515	FastAPI app, `Protection` middleware, `case_json`, every endpoint, `run_tool`, `build_chart`

14.3 Database schema (SQLite)

Table	Columns	Notes
`departments`	code PK, name, name_kn, color, pilot	38 rows; pilot = REV, RDPR, FOOD

Table	Columns	Notes
`line_departments`	id PK, dept_code, name, name_kn, n_real	
`services`	id PK, line_id, dept_code, name, full_name, name_kn, sakala, n_real, **role, role_share, role_support**	124 pilot services with learned routing roles
`grievance_types`	id PK, service_id, name, n_real	221
`officers`	id PK, name, short, dept_code, level, role, taluk, gp, up_id, data_name, seen_in_data, n_real	88
`gazetteer`	village, gp, taluk, district, n_real; PK (village, gp, taluk)	198 villages
`ipgrs_records`	gid PK, text_masked, dept, dept_code, line, service, grievance, service_type, district, taluk, gp, village, created, status, role, officer, officer_role, source, service_id	9,999 masked rows
`sakala_services`	key PK, name, name_kn, dept_code, days, do_role, co, aa, checklist	3 illustrative services
`cases`	id PK, kind, text, masked, summary, src, lang, district, taluk, gp, village, dept_code, line, service, service_id, grievance, conf, why, engine, vulnerable, vuln_reason, owner, level, level_since, created, status, real, synthetic, route_basis, sakala (JSON), atr, atr_score, atr_reason, session, linked_to, **audit** (JSON, added by `migrate()`)	715 at go-live
`case_events`	id, case_id, t, who, what, color, note	The audit trail
`notifications`	id, officer_id, t, msg	Officer inbox badge
`chat_sessions`	id PK, state (JSON), updated	WhatsApp session state
`chat_messages`	id, session, t, me, payload	Transcript log
`citizen_msgs`	id, case_id, session, t, officer_id, kind, text, delivered	Officer → citizen outbox
`ai_log`	id, t, task, engine, ms, ok, detail	Every model call
`kv`	k PK, v (JSON)	`now`, `seq` (from 504200), `model_eval`, `routing_eval`, `low_conf`
`sakala_perf`	district, month, receipts, disposals, rejected, delayed, intime, overdue, pop, total_w, rank, source; PK (district, month)	384 rows, created by `analytics.load_perf`

Indexes: `cases(owner)`, `case_events(case_id)`, `ipgrs_records(taluk)`. Case ids: `GR-<seq>` grievances, `GR-<gid>` imported real cases, `SK-RD855S26…` / `SK-FD201S26…` applications.

14.4 Request lifecycle and concurrency

- Uvicorn, single process. `Protection` (pure ASGI) checks `content-length` and the per-IP minute window, then adds the security headers to every response.
- All DB access goes through one connection guarded by an `RLock`; the gateway client is one shared `httpx.Client`; `ai.parallel` uses an 8-worker thread pool for evaluation runs only.
- Startup: `db.conn()` (schema + migrate), create the pristine snapshot if missing, `local_ai.load()` (trains if the joblib is missing).

- Reset: `sqlite3.backup()` from the pristine file into the live connection (works while the file is open on Windows), then schema + migrate again and clear analytics caches.

14.5 Known code findings (for the rebuild)

- `ingest.seed_cases` inserts 35 positional values into a 36-column `cases` table after `migrate()`; a fresh `python -m server.ingest` should use an explicit column list.
- `check_atr` discards the model's `reopen` flag; `create_application` ignores `sakala_services.aa` and always uses `dc_mys`.
- `alternatives` in the audit is dead code for auto-routed cases.
- The Sakala-breach event does not queue a citizen message.
- The generated UI lacks the click handler for "Route it"; the server endpoint works.

15. Complete API reference

Base URL `http://localhost:8020`. JSON in, JSON out; no authentication; FastAPI returns 422 for a malformed body and 500 for an unhandled error. **This is the whole surface Jarvis needs to drive the three profiles.** Shapes below are copied from `api.py`.

15.1 Bootstrap and state

Method, path	Request	Response	Notes
`GET /api/bootstrap`		`officers {id: {n, short, d, lv, up, role, taluk, gp, seen, dataName, nReal}}, geo {Mysuru: {taluk: [gp…]}}, depts {code: {name, kn, c}}, now, sla, ai {llm, model, asr, asr_model, reason}, evals {routing, model, llm}, masters {services, grievanceTypes, officers, villages, records, gps}, samples [[label, text]]`	Load once
`GET /api/state?session=&scope=all|citizen`		`{now, cases: [case], notifs: [{to, t, m}] (latest 400; empty for citizen scope), ai}`	`scope=citizen` returns only that session's cases
`GET /api/health`		`{ok, time (wall clock), ai, local_model}`	
`GET /api/ai_log?limit=200`		`{total, failures, rows: [{id, t, task, engine, ms, ok, detail}]}`	Failures first; limit 10–1000

Case object (`case_json`): `id, text, masked, summary, src, district, taluk, gp, village, owner, level, levelSince, created, status, vulnerable, real, synthetic, engine, routeBasis, atr, atrScore, atrReason, linkedTo, mine, audit {…}, u {d, line, svc, gr, conf, why, type, vulnerable, vulnReason}, sakala {key, service, dueDays, DO, CO, AA, complete, applicant, illustrative, breached?} | null, history [{t, who, what, c, note}]`. Status values: `Pending`, `In progress`, `Info requested`, `Scrutiny`, `Resolved`, `Rejected`. Days left are computed client-side from `levelSince` / `sakala.complete` and `sla` (see 12.1).

15.2 Citizen chat

Method, path	Request	Response
`POST /api/chat`	`{session?, action?, text? (≤ 2000), image_b64?, media_type? ("image/png"), voice_text?, audio_b64?}`	`{session, events: […]}` (always 200; server errors arrive as an apology message plus a toast)

Actions and event shapes are in section 9.3 and 9.6. Rate limit 120 per minute per IP. A new session is created when `session` is empty; keep the returned id. `action:"poll"` returns only newly delivered officer messages.

15.3 Officer and CS actions

Method, path	Request	Response	Errors
`POST /api/cases/{id}/action`	`{action, officer?, note?, reason?, atr?, to?}`	`{ok, toast: [title, text], …}`; `resolve` adds `{score, reason, engine}`	404 case not found · 400 unknown officer / unknown action / reason mandatory / ATR too short / compensation only on Sakala / choose the officer · 403 not the owner / not the Competent Officer / L1 cannot re-route · 409 case is {status} / only scrutiny cases re-route / applications never re-route / nothing to compensate
`POST /api/cases/{id}/draft_atr`		`{text, engine: "Sakala AI" | "template", error?}`	404
`POST /api/clock/tick`	`{days}` (1–30)	`{now, escalated, breached}`	
`POST /api/reset`		`{ok, now}`	

Actions: `accept`, `info`, `return`, `reject`, `resolve`, `reroute`, `nudge`, `comp_order`, `comp_hrms` (rules in section 10.3). Send `officer: ""` to act as the Chief Secretary's office (allowed for `nudge` and `reroute` only).

15.4 AI mode and voice

Method, path	Request	Response
`POST /api/ask`	`{q}` (≤ 500 chars)	`{engine: "ai", html, tools: [names], chart: {type: "bar"|"line", labels, full?, values, series_label, unit, title, source} | null}`; or `{engine: "local"}` when the gateway is down or `q` is blank (the UI then answers locally)
`POST /api/transcribe`	`{audio_b64, lang: "en"|"kn"}` (16 kHz mono 16-bit WAV)	`{text}`; `{text: "", error}` on failure (always 200)

`run_tool` tools the planner may call (`ai.ask` picks at most 3):

Tool	Args	Returns
`query_cases`	`dept_code`, `taluk`, `status`, `service_contains`, `officer_contains`, `overdue_only`, `group_by` ∈ officer / service / taluk / level / status / source / dept / none	grouped: `{total, groups: [[label, n]] (top 15)}`; ungrouped: `{total, open, sample: [{id, service, officer, status, taluk, gp, days_left}] (10)}`
`real_records`	`group_by` ∈ dept / service / district / source / status / role, `district`, `dept_code`	`{groups, total}` over the 9,999 rows
`sakala_2025`	`district?`	`{state, worst_late (8), best_late (5), monthly}` or `{state, district, monthly_state}`; `monthly = [{month, overdue, late_pct, rej_pct}]`

Tool	Args	Returns
`insights`	`kind` ∈ hotspots / scrutiny / vulnerable / escalations	hotspots `{open, week}` ; scrutiny `{source: n}` ; `{open_vulnerable}` ; `{auto_escalated_cases}`

`build_chart(question, results)` turns the first chartable result into the chart: a monthly line when the question says trend / month / over time (rejected %, late % or overdue by keyword), else districts by late % (bar), else any `groups` of ≥ 2 (bar, ≤ 12 labels), else hotspots or scrutiny-by-channel bars.

15.5 Intelligence (GET)

Path	Returns
`/api/insights/quality`	`{total, checks: [{k, n, fix}] × 9}`
`/api/insights/misrouting`	`{n_checked, n_flagged, kridl_general, by_source, examples: [{gid, text, filed, suggested, confidence, officer, district, source}]}`
`/api/insights/historical?summary=1`	`{total, pilot, camp_share, by_day, biggest_day, biggest_day_n, sources, scrutiny_stuck, scrutiny_rate_app, top_issues_by_taluk, vulnerable_waiting, repeat_complaints, top_services_state, summary?}`
`/api/insights/hotspots?fix=1`	`{open: [{service, gp, taluk, n, v, ids}], week: […], fixes?: [{place, root_cause, fix}]}`
`/api/insights/accuracy?run=1&n=40`	`{routing {n, agree, rate, top_misses}, local_model {dept_acc, service_acc, service_top3, …}, llm {n, errors, threshold, auto_routed_share, dept_acc, service_acc, officer_acc, seconds, model, at}}` ; `run=1` re-runs the live test and re-tunes `kv.low_conf`
`/api/sakala/tracker`	`{state {receipts, late_pct, rej_pct, overdue_first, overdue_last, months, districts}, monthly, districts [{district, receipts, disposals, rejected, delayed, late_pct, rej_pct, overdue_last, rank_last}], source}`
`/api/sakala/early_warning`	`{districts [{district, late_now, late_before, rej_now, rej_before, overdue_now, overdue_before, rej_trend, late_trend, official_rank, official_score, flags, overdue_pl, fair_score, fair_rank}], flagged, official_spread, fair_spread}`
`/api/review_pack?level=cs|dc|ac|eo&officer_id=`	`{facts {as_of, scope, open, overdue, vulnerable_waiting, auto_escalated, top_issues, most_overdue_officers, sakala_deadlines_missed, stuck_examples, sakala_2025}, html, kn}`
`/api/e4`	`[{officer, id, defaults}]` for more than 7 overdue

15.6 Upload and static

Path	Notes
`POST /api/upload` (multipart `file`)	`.xlsx/.xls/.csv`, 5 MB max (413). Returns `{kind: "performance_report", rows, months, districts}` or `{kind: "complaints", rows, routed, results: [{text, masked_items, intent, engine, case_id?, officer?, service?, dept?, conf?, gp?, taluk?, linked_to?, note?}]}`; 400 when no description column
`GET /`	`web/index.html`
`GET /assets/*`, `GET /samples/*`	static
`GET /dbg`	`web/dbg.html` if present (test harness), else 404

15.7 Rate limits and headers (middleware)

`/api/transcribe` 240 · `/api/chat` 120 · `/api/ask` 60 · `/api/cases/*` 120 · `/api/upload` 20 · `/api/clock/*` 60 requests per client IP per rolling minute → 429 `{detail: "Too many requests…"}` with `retry-after: 60`; `content-length` over 8,000,000 → 413. Every response carries `x-content-type-options: nosniff`, `x-frame-options: DENY`, `referrer-policy: no-referrer`, `permissions-policy: camera=(), microphone=(self), geolocation=()`. **For a voice agent polling or streaming at high frequency, raise the `/api/chat` and `/api/cases` limits or move to server-sent events (section 17).**

16. UI build system, design tokens, tests and operations

The app UI is not written by hand: `tools/make_app_ui.py` takes the clickable wireframe (simulated AI, state in the browser) and patches it into the live app (API-driven). Make visual changes in the wireframe, then regenerate; editing `app/web/index.html` directly is lost on the next build.

16.1 Build

```
cd tools
python make_app_ui.py ../wireframe/index.html app_wa_client.js ../app/web/index.html
app_insights.js
```

The script empties the simulated `OFF` / `GEO` data (they come from `/api/bootstrap`), replaces `seed()` with `boot()` / `loadState()`, swaps the citizen module for `app_wa_client.js`, routes drawer actions to `/api/cases/{id}/action`, adds the CS navigation items, includes Chart.js, then concatenates `app_charts.js` and `app_insights.js`. The wireframe's history of patch scripts (`patch_wa.py`, `patch_future2.py`, `patch_twoway*.py`, `patch_logo.py`, `patch_map*.py`, `patch_refine.py`, `patch_guardrails.py`, `patch_arch.py`) has already been applied; backups are in `tools/backups/`.

Front-end file	Role
`wireframe/index.html`	Shell, all CSS (future2 layer at about lines 705–1139), officer and drawer (1650–1764), CS and AI mode (1766–1887), map (1899–2059), profiles and `goAI` (2063–2091)
`tools/app_wa_client.js` (191 lines)	WhatsApp client: queue, `playEvents`, polling, officer bubbles, chips, list sheets, WAV recorder (`encodeWav` to 16 kHz mono), photo upload, `waPing`, `fitWA`
`tools/app_insights.js` (122)	Insights, Accuracy, Sakala tracker, Wednesday pack, Upload, Roadmap, `askVoice`, `safeHtml`
`tools/app_charts.js` (138)	`renderChart` (Chart.js), `svgChart` fallback, `chartFromHbar`
`tools/future2.css` (434)	The Apple / OpenAI-grade layer: tokens, pill buttons, ink heroes, glass panels, AI mode, map, WhatsApp fit, charts, audit drawer

16.2 Design tokens (from `future2.css :root`)

Group	Values
Neutrals	ink `#0b0b0c`, ink-2 `#1d1d1f`, mute `#6e6e73`, canvas `#f5f5f7`, surface `#ffffff`, lines `rgba(0,0,0,.08 / .12)`
Emerald (primary)	`#0e9f6e`, `#16c784`, ink `#065f46`, soft `#e7f8f0`
Saffron (accent, sparing)	`#f26b1d`, `#ff9a3c`, soft `#fff2e8`
Status	good `#12a150`, warn `#c88a04`, bad `#e5484d`, blue `#0a84ff`, violet `#7c5cff`
Karnataka flag (map)	yellow `#ffcd00`, red `#c8102e` / `#d82221`
Departments	Revenue `#26732f`, RDPR `#3b7dd8`, Food `#da6823`
Taluks	Piriyapatna emerald `#16c784`, Hunsur blue `#3b9bff`, Mysuru violet `#9b7bff`

Group	Values
Chart palette	`#0e9f6e #f26b1d #3b82f6 #8b5cf6 #16c784 #ff9a3c #0ea5e9 #d6409f #64748b #14b8a6 #f59e0b #6366f1`
Fonts	Inter 400–800, Noto Sans Kannada; letter-spacing −.011em; features cv11, ss01, ss03
Shape	radius 20 (cards), 999 (pills), 24–26 (drawer, modal, heroes); soft two-layer shadows

Components: black pill primary buttons, saffron gradient for compensation steps, dark ink heroes with emerald / saffron / blue glows, frosted header and sidebar (`backdrop-filter: blur(20px)`), badges `b-g / b-b / b-s / b-n / b-r / b-w`, dashed-border `prodtag` for PRODUCTION PHASE / MOCK HRMS, `.ai-note` gradient boxes for AI-written text. The official Sakala logo (`assets/sakala-logo.svg`, a vector redraw of `sakala.kar.nic.in/img/Sakala_logo.jpg`) appears in the favicon, splash, header, WhatsApp avatar, AI-mode hero and as a 5.5% watermark; always beside `karnataka-emblem.png`.

16.3 Global behaviours worth keeping

- Splash ("ಸಕಾಲ · Sakala AI · Right officer · First time · On time") fades after 1.3 s; the "Entering AI mode" transition spins the logo halo for 1.25 s.
- Header: "Government of Karnataka · Praja Seva Department", "ಸಕಾಲ Sakala AI · Citizen Request Intelligence", pill "LIVE · MYSURU PILOT", demo clock (+1 day, +7 days, ↺ reset), English / ಕನ್ನಡ (the Kannada toggle only toasts; full Kannada UI is production phase), a slim ink app bar with only the left text ("Citizen · Sakala AI on WhatsApp", "Officer · {name}", "Chief Secretary · Mysuru pilot"). The ticker, engine tag and AI MODE badge are hidden by CSS.
- Bottom-left **profile switcher** (Citizen / Officer / Chief Secretary). Every switch calls `goHome()`: Citizen → WhatsApp, Officer → inbox, CS → Command dashboard, never AI mode; state reloads with the right scope; the WhatsApp history survives in `sessionStorage` (`wa_sid`, `wa_hist`, `wa_stage`).
- Deep links: `?p=citizen|officer|cs&v=<view>&off=<officerId>&open=<caseId>`; "Open in officer's inbox" from the citizen result card; officer rows on the CS dashboard.
- `countUp()` animates every KPI on render; toasts are dark pills with emerald titles, 3.6 s.

16.4 Tests (all green on 14 Oct)

Suite	Run	Covers
`tools/tests/test_backend.py`	from `app/`: `PYTHONIOENCODING=utf-8 python ../tools/tests/test_backend.py [t_name…]` with the server up	Routing matrix; understanding (16 crafted cases, 16/16); every chat path (34); officer permissions; clock and escalation; intelligence endpoints incl. a live accuracy run; upload + AI mode. Chat / officer / clock sections 59/59
`tools/tests/test_twoway.py`	same	Officer → citizen messages, replies, re-open, Kannada translation (19/19)

Suite	Run	Covers
`tools/tests/test_guardrails.py`	same	Injection, off-topic, sanitising, vision and voice fail-safes, session isolation, audit, rejection + appeal, ladders, masters, threshold, rate limit, headers, 413 (27/27)
Browser flows `tools/app_tests/run_app.sh` + `hook_app.txt`	headless Edge with `--virtual-time-budget`; copy `app/web/index.html` to `app/web/dbg.html` with the hook injected before `</body>`, run `bash tools/app_tests/run_app.sh "p=citizen&flow=water"`, delete `dbg.html` afterwards	Flows: views (all profiles), water, light, apply, photo, voice, resolve, twoway, aimode, charts, map cats, profile switch; `window.onerror` and unhandled rejections are captured as `data-err`

`tools/tests/asr_test.wav` is a real clip for the speech test. The scripts carry hard-coded temp paths from the original machine; adjust them.

16.5 Running, resetting, Windows notes

- Start: `app\run.bat`, or from `app\`: `python -m uvicorn server.api:app --port 8020`; open `http://localhost:8020`. Python 3.11.
- The gateway needs the office LAN or VPN; without it the app runs on the offline model and the header engine label changes.
- Rebuild everything from the Excel: stop the server first (the DB is locked on Windows), then `python -m server.ingest` (about 2 minutes; recreates the DB, the pristine copy and the model).
- Reset the demo: the ↺ button or `POST /api/reset`.
- Ports 8000 and 8001 on the build laptop belong to other apps (`app.main:app`); only stop processes whose command line contains `server.api:app`. The process name is `python3.11`.
- Use `PYTHONIOENCODING=utf-8` for Kannada output; bash heredocs with backticks break on Windows, so write scripts to files; headless screenshots to paths with spaces fail.
- Keep `wireframe/` and `app/web/index.html` visually in sync (section 11 of `CLAUDE.md`).
- The sister projects `d:\KPCL POC` and `d:\RDPR POC` are strictly read-only reference material (their `.env`, Chart.js vendor file and visual style were copied from, never edited).

17. The Jarvis brief

Goal. The same Sakala POC, driven entirely by voice across all three profiles, with live synchronisation between screens and a visual stage that looks like a 2026 product launch. The Chief Secretary's own words after the demo: it works, but there is "no wow factor". Jarvis is the wow factor; the underlying system (sections 2 to 16) must keep working exactly as it does, because that is what she already trusts.

17.1 Decisions already taken

Decision	Value	Why
Language	**English only** (voice in and out). Kannada text in the chat stays as it is	`qwen3-asr` and `qwen3-tts` on the gateway have no Kannada; adding Kannada would need extra models deployed or an outside vendor
Models	**Only the existing `.env` gateway on the team's H200**: `qwen3-asr` (listen), `qwen4-deploy` (think), `qwen3-tts` (speak)	Everything stays on government-controlled hardware; no new keys or vendors
Interaction	No clicking, tapping or typing in the demo path. Jarvis greets ("Hi, how may I help you?"), raises, locates, confirms, tracks, replies, resolves, nudges and answers	The CS's request
Conversations	Voice clips flow **citizen** ↔ **officer** and **officer** ↔ **CS**, never citizen ↔ CS	Mirrors the real chain of accountability
Officer experience	"Peaceful and enjoyable": the officer hears the inbox, speaks the ATR, hears the AI score, done	The officer is the bottleneck the system exists to help
Build	The R&D team builds the whole thing from this document and the code; refinements after	Team lead's decision

17.2 Target experience, profile by profile

Citizen (WhatsApp stage). Jarvis speaks the greeting; the citizen says "there is no drinking water in Gorahalli"; the pipeline panel lights up stage by stage while Jarvis narrates ("I understand this as Rural Development, drinking water, 93% sure … Gorahalli is in Chapparadahalli Gram Panchayat, Piriyapatna … shall I file it?"); the citizen says yes; Jarvis reads the Registered card ("Assigned to the AE for Rural Drinking Water, Piriyapatna. Action is due in 7 days, then it moves to the Executive Officer"). Later the officer's voice reply arrives as a voice bubble; Jarvis asks "was your problem solved?"; the citizen answers yes or no.

Officer. "Good morning, Tahsildar. You have 12 open cases, 2 overdue, 1 vulnerable citizen waiting. Shall I start with the overdue ones?" Jarvis reads the masked complaint, the AI classification and the routing evidence, then takes dictation: "Inspected the site on the 15th, pipeline repaired, supply restored". Jarvis files it, the AI scores it 5/5, and the citizen is told in their chat, by voice clip and text. The officer can also say "ask the citizen which street", "this is not my jurisdiction", "reject: duplicate of an existing request", or, as Competent Officer, "order the compensation".

Chief Secretary. "Madam, 393 cases are pending in the pilot, 1 overdue, 3 vulnerable citizens waiting. Overdue in the state fell 63% this year; Bengaluru Urban still holds a third of it." The map glides to Mysuru as she speaks; "show me the overdue ones" lights the Overdue tile; "which officer has the most overdue cases?" runs AI mode and the chart morphs in; "nudge the Tahsildar" fires the nudge; "send the Tahsildar a note" records an officer-bound voice clip. The Wednesday pack is read aloud on request.

17.3 Proposed stack (all on the H200 gateway)

Layer	Choice	Notes
Microphone and turn-taking	Browser Web Audio + **Silero VAD** in the page (same pattern as the KPCL voice app, copy only)	Detects end of speech in about 0.3 s; allows barge-in (interrupt Jarvis mid-sentence)
Listen	`qwen3-asr` via `POST /api/transcribe` (already built; 16 kHz WAV)	About 0.4 s per utterance; send chunks on VAD end-of-speech
Think	`qwen4-deploy` as an **intent router with guided JSON**: schema `{intent ∈ raise / locate / confirm / track / reply / feedback / appeal / open_case / accept / ask_info / resolve / return / reject / nudge / reroute / comp_order / ask_data / navigate / help, args}`; then the existing endpoint is called	Same enum trick as `understand()`; the model never free-generates actions
Speak	`qwen3-tts` (`TTS_API_STYLE=openai`, `TTS_STREAM_ENABLED=true`, voice `serena`, reference WAV) via a new `POST /api/speak {text}` that streams audio	Pre-render greetings and fillers ("one moment", "filing it now") for zero-latency starts
Live sync	**Server-sent events** `GET /api/events` fed by `case_events`, `notifications` and `citizen_msgs` inserts, replacing the 3.5 s poll	All three screens update the moment anything changes
Voice clips between roles	New table `voice_clips(id, case_id, from_role, to_role, officer_id, session, audio_path, transcript, t, heard)`; the AI works on the transcript (ATR scoring, masking, audit), the human hears the clip; delivered as a new `msg` type `voice` in `deliver()` and as a drawer item for officers and the CS	Keeps every existing guardrail working on the text
Visual stage	Voice orb (reactive to input and output amplitude), live captions, the existing pipeline panel, map glides and Chart.js morphs; GSAP or Three.js for the orb and transitions	All in-house, hides latency
Fallbacks	ASR fails → the text box; TTS fails → browser `speechSynthesis` + captions; gateway down → offline model and scripted replies	The demo must never stall on stage

Expected turn latency: VAD 0.3 s + ASR 0.4 s + intent 0.3–0.7 s + action 0.1 s + first TTS audio 0.3–0.6 s ≈ **1.4–2.1 s**. Pre-rendered fillers make it feel under 1 s.

17.4 Mapping Jarvis intents onto the existing API

Spoken intent	Call	Spoken result from
Raise a grievance	`POST /api/chat {text}` (or `{action:"voice", audio_b64}`)	the bot `msg` and `stage` events
Share location / pick GP / typed village	`{action:"loc:share"}`, `{action:"gp:<GP>|<Taluk>"}`, `{text:"<village>"}`	confirm card
Confirm / change	`{action:"submit"}` / `{action:"change"}`	Registered card, `result`
Track	`{action:"track"}`	the track card rows

Spoken intent	Call	Spoken result from
Reply to officer, feedback, appeal	`{action:"reply:<cid>"}` then `{text}`; `{action:"fb:<cid>:yes|no"}`; `{text:"appeal"}`	confirmation messages
Officer inbox	`GET /api/state` filtered by `owner` and status, sorted vulnerable first then least `left`	KPIs and rows
Accept / ask / resolve / return / reject / compensation	`POST /api/cases/{id}/action`	`toast` and, for resolve, `score` + `reason`
Draft the ATR	`POST /api/cases/{id}/draft_atr`	`text`
CS summary	`GET /api/review_pack?level=cs` facts (plus `html` if the gateway is up)	spoken facts
Ask the data	`POST /api/ask {q}`	`html` (strip tags) + `chart` rendered on screen
Nudge / re-route	`action: nudge` / `reroute` with `officer: ""`	toast
Move time	`POST /api/clock/tick {days}`	`escalated`, `breached`

Two things to add server-side: `/api/speak` (TTS proxy with streaming) and `/api/events` (SSE). Raise or bypass the `/api/chat` rate limit for the voice session.

17.5 Guardrails specific to the voice layer

- Voice never bypasses the enum: an unrecognised request is answered "I can help with grievances, applications, tracking and your cases", never executed.
- Destructive or irreversible actions (reject, resolve, compensation, re-route) get a spoken confirmation ("Reject GR-504203 as a duplicate. Confirm?").
- The officer id is bound to the session at "log in as"; Jarvis repeats who it is acting as.
- Transcripts go through `privacy.mask()` before any model call and before display; clips are stored, transcripts are what the AI sees.
- Barge-in stops TTS immediately; the orb shows listening / thinking / speaking states so nobody talks over the system.

17.6 Build plan (two working days)

1. **Day 1 morning:** `/api/speak` with `qwen3-tts` streaming; `/api/events` SSE; test both from the existing UI. Verify `qwen3-asr` accuracy on 10 Indian-English sentences recorded in the demo room; if more than 2 of 10 are wrong, tune the ASR prompt hints (`ASR_CONTEXT_HINTS_ENABLED`: place and service names).
2. **Day 1 afternoon:** the intent router (guided JSON) and the citizen flow end to end by voice; the orb and captions.
3. **Day 2 morning:** officer and CS flows; voice clips between roles; map and chart hooks.
4. **Day 2 afternoon:** an 8–10 line rehearsed script across citizen → officer → CS, 20 dry runs, the fallbacks, and a pre-recorded backup video.

17.7 Acceptance criteria

- A citizen files, locates and confirms a grievance by voice alone, and hears the officer's reply as a voice clip.

- An officer clears an overdue case by voice (dictated ATR, AI score read back, citizen notified) without touching the keyboard.
- The CS asks three questions by voice and sees the map, the tiles and a chart respond; a nudge reaches the officer's screen within a second.
- Every action still appears in the audit trail and the AI log; the guardrail tests still pass.
- All models called are the three in the existing `.env`; no new vendor.

17.8 Honest risk score

As assessed on 14 Oct: **8 / 10** achievable in two days on the H200 with the stack above (voice quality 8, understanding of Indian English 8, speed 8, visual wow 8, reliability 7.5). The single real risk is the venue network: the gateway is only reachable on the office LAN or VPN, so plan a hotspot with VPN and the fallbacks in 17.3. Rehearsal and barge-in are what move it towards 9; open-ended chatter is where it breaks.

18. Glossary, fact sheet and sources

18.1 Glossary

Term	Meaning
Sakala	Karnataka's legal guarantee of government services within a notified time (Act 1 of 2012)
GSC number	Sakala application number (from the Act's original name "Guarantee of Services to Citizens"), e.g. `RD855S260004811`
iPGRS / Janaspandana	Integrated Public Grievance Redressal System, the state grievance portal (live since 1 Nov 2021), helpline 1902
Praja Seva Department	New department (June 2026) merging Sakala Mission, Public Grievances and Administrative Reforms
Praja Seva Abhiyana	Saturday grievance camps; 41% of the real sample
LMF / L1, L2, L3	Last Mile Functionary (first-level field officer), then the two escalation levels above
DO / CO / AA	Designated Officer, Competent Officer, Appellate Authority on a Sakala application
ATR	Action Taken Report, mandatory before closing a grievance
Form C	Sakala notice to recover compensation cost from the officer at fault (s.11)
Form E-4	Sakala show-cause notice (used here for officers with more than 7 overdue cases)
PDO	Panchayat Development Officer, one per Gram Panchayat
EO TP	Executive Officer, Taluk Panchayat (L2 for RDPR)
AEE PRED	Assistant Executive Engineer, Panchayat Raj Engineering Division (roads)
AE RDWS	Assistant Engineer, Rural Drinking Water & Sanitation
ADLR / DDLR	Assistant / Deputy Director of Land Records (survey)
AC / DC	Assistant Commissioner (sub-division) / Deputy Commissioner (district)
ZP CEO	Chief Executive Officer, Zilla Panchayat (L3 for RDPR)
GP	Gram Panchayat, the village-cluster local body
Taluk	Sub-district; pilot taluks Piriyapatna, Hunsur, Mysuru
Scrutiny desk	The human queue for low-confidence or returned cases
Demo clock	The simulated time (`kv.now`) that drives deadlines
Gateway	The team's internal OpenAI-compatible vLLM server on the H200
Guided JSON	vLLM `response_format: json_schema` ; enums physically constrain the model's choices

18.2 Fact sheet (quote these)

- 9,999 real iPGRS grievances, 3–9 Oct 2026; 38 departments, 195 line departments, 703 services, 838 grievance names; 14% Sakala-tagged.
- Channels: camps 41%, CSC 24%, web 19%, app 9%, call centres 6%. 39% of app complaints stuck in scrutiny.

- Pilot: Revenue, RDPR, Food; Mysuru district; Piriyapatna, Hunsur, Mysuru taluks; 124 services, 221 grievance types, 88 officers, 61 GPs, 198 villages; 703 real live cases.
- Routing proof: 97.6% officer agreement on 673 real cases; end to end from text: 97.5% department, 95% service, 92.5% officer (n = 40).
- Offline model: 71% department, 39% service (hold-out).
- Sakala 2025: 35.2M receipts, 4.45% late, 7.42% rejected, overdue 51,172 → 19,129 (−63%); Bengaluru Urban worst (9.2% late), Kalaburagi best (1.9%), Mysuru 5.4%.
- Law: ₹20/day up to ₹500 (s.9); Form C recovery (s.11); appeals 30 / 60 days (s.12–13); more than 7 defaults → enquiry (s.14(4)).
- Assumed: grievance ladder 7 / 5 / 3 days; Sakala timelines 21 / 45 / 30 days; go-live clock for imported cases; number-only masking.
- Demo: 14 Oct 2026, 3 PM, Chief Secretary Shalini Rajneesh; outcome "correct, no wow factor"; Jarvis is the next step.

18.3 People

Pranay (build lead, siddharthpranay@gmail.com) · Mohan (lead, go / no-go) · Pallavi (masters and hierarchy) · Ram (data, DB, dashboard) · Malikarjuna (voice and R&D) · Thulasi Ram (scope document). Framing: "Neural AI Governance", connect rather than replace.

18.4 Sources

- Sakala portal: sakala.kar.nic.in (Act, Rules and amendment PDFs under `/pdf/`); Act text at indiankanoon.org/doc/14397538.
- NeGD directory: iPGRS negd.gov.in/isl/Directory/statedata/407, Janaspandana /418, Sakala /397.
- iPGRS portal: ipgrs.karnataka.gov.in; department list: karnataka.gov.in/department/en.
- Press: Deccan Herald, 11 Sep 2026, on AI routing of Praja Seva grievances.
- Project files: `Sakala_POC_Scope.pdf`, `Sakala Use cases and Important Links.pdf`, `Sakala_AI_POC_Final_Use_Cases.pdf`, `Sakala_Explained.md`, `Sakala_AI_POC_Architecture.png/.pdf`, `CLAUDE.md`, the two client architecture images (note: they place Bylakuppe in Hunsur; it is in Piriyapatna), and `AI Grievance Assistant – POC Architecture.pdf` (team lead's document).
- Map data: KGIS district outlines; OpenStreetMap taluk polygons (`tools/osm/`).